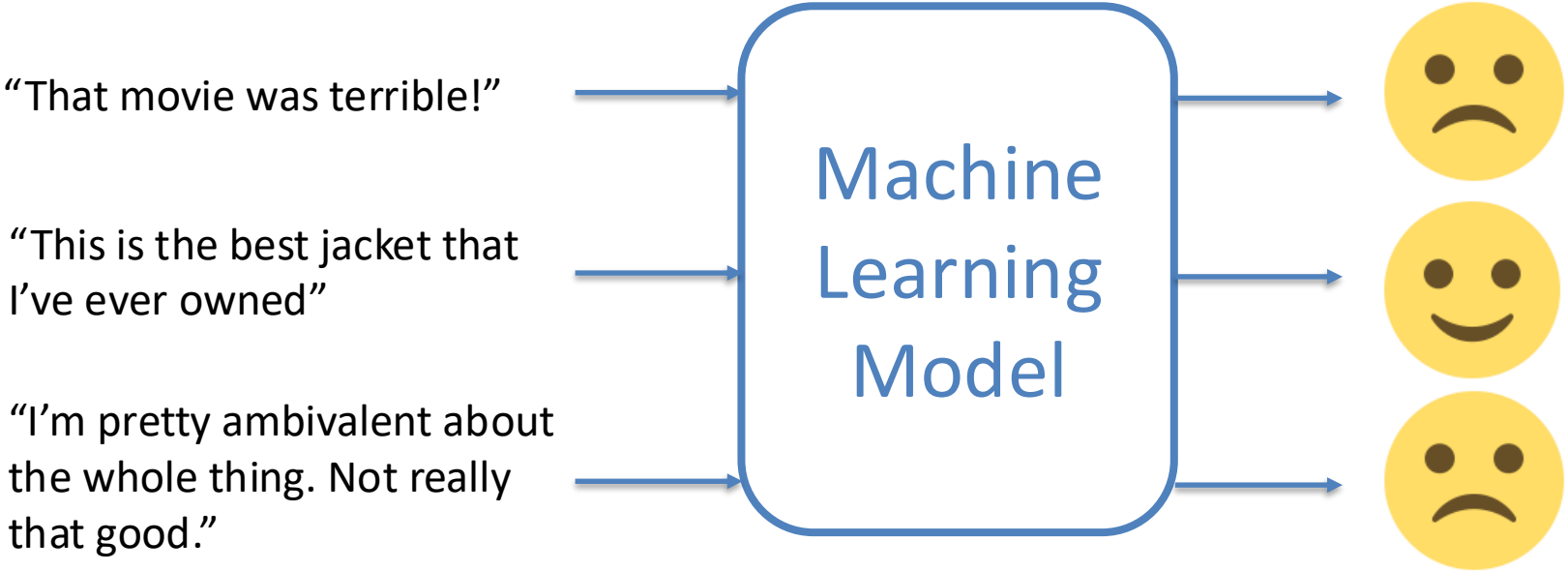
ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 5
01/27/2026

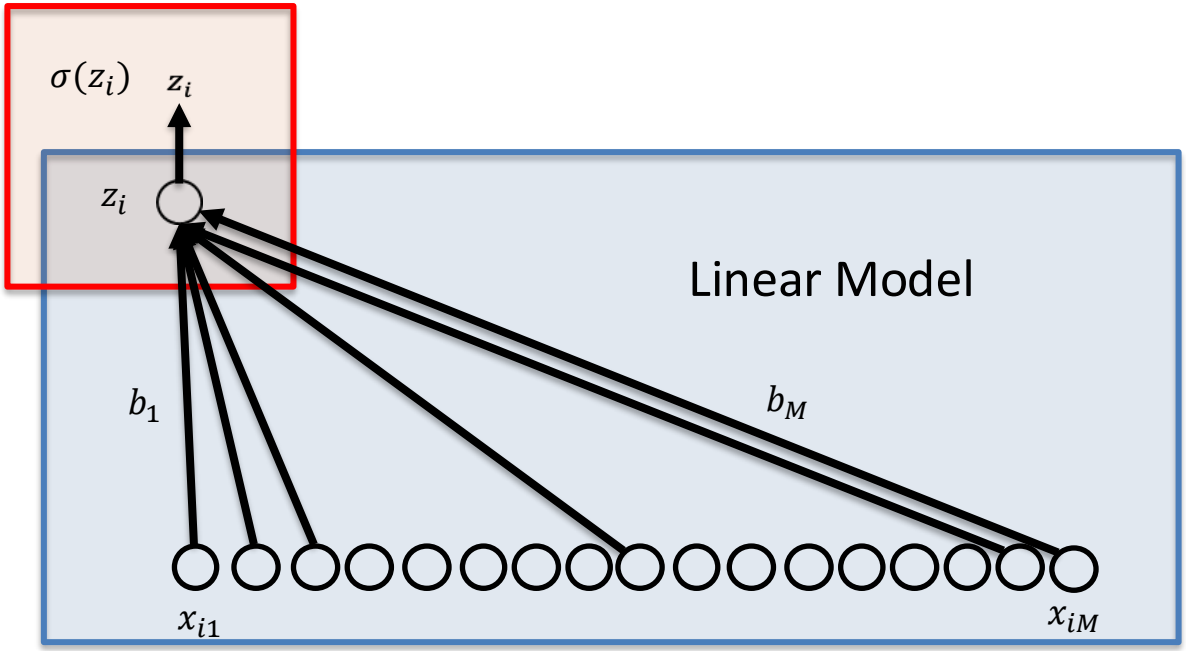
Sentiment Analysis



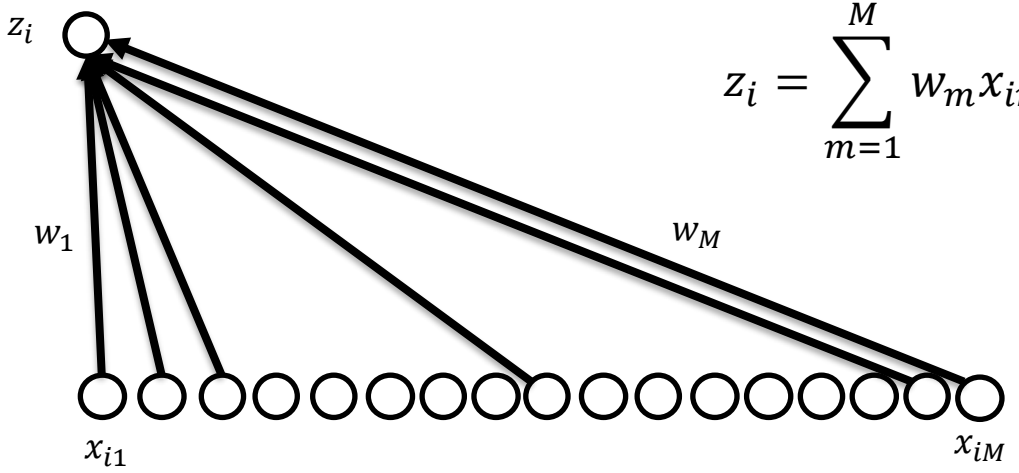
This is a **binary classification**, just like logistic regression!

Revisiting Logistic Regression

Convert to Probability



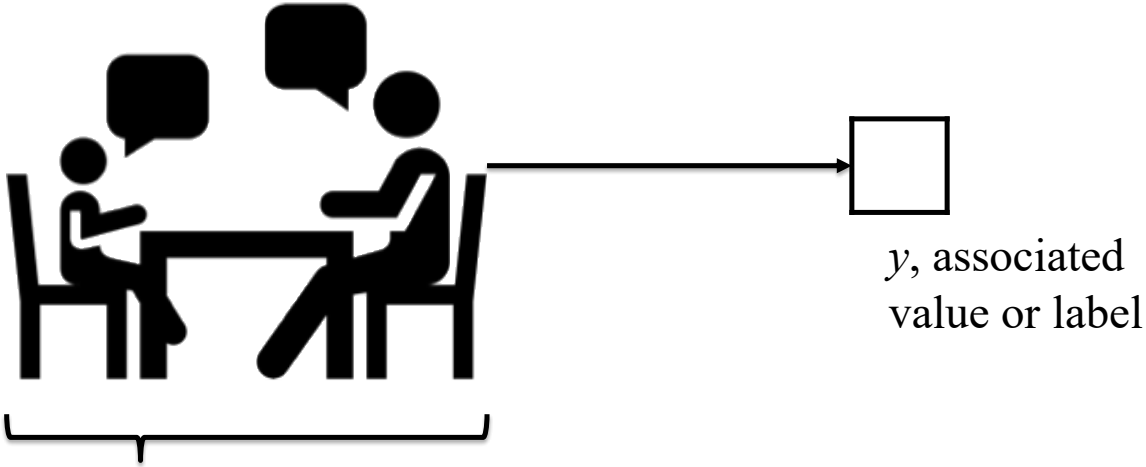
Revisiting our Linear Predictive Model



$$z_i = \sum_{m=1}^{M} w_m x_{im} + b$$

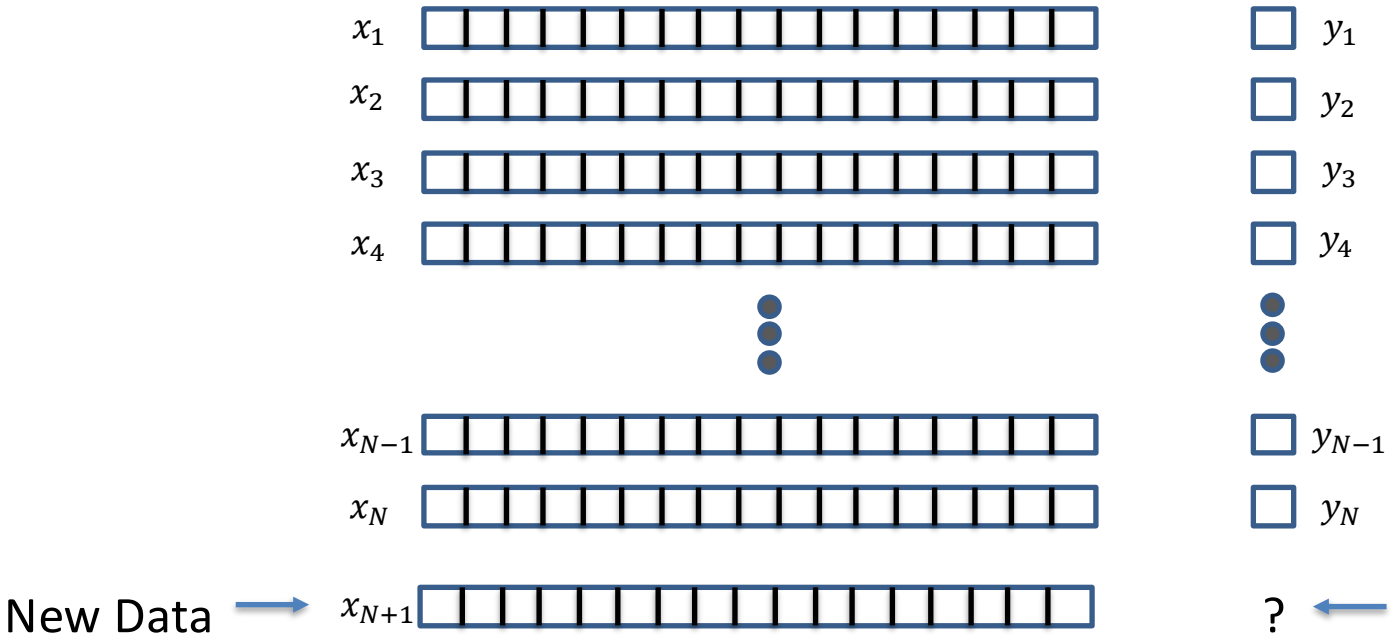
Big issue: we do not have a vector of features…

Language in a predictive model?



End goal: predict y from x

Our Previous Training Set Setup

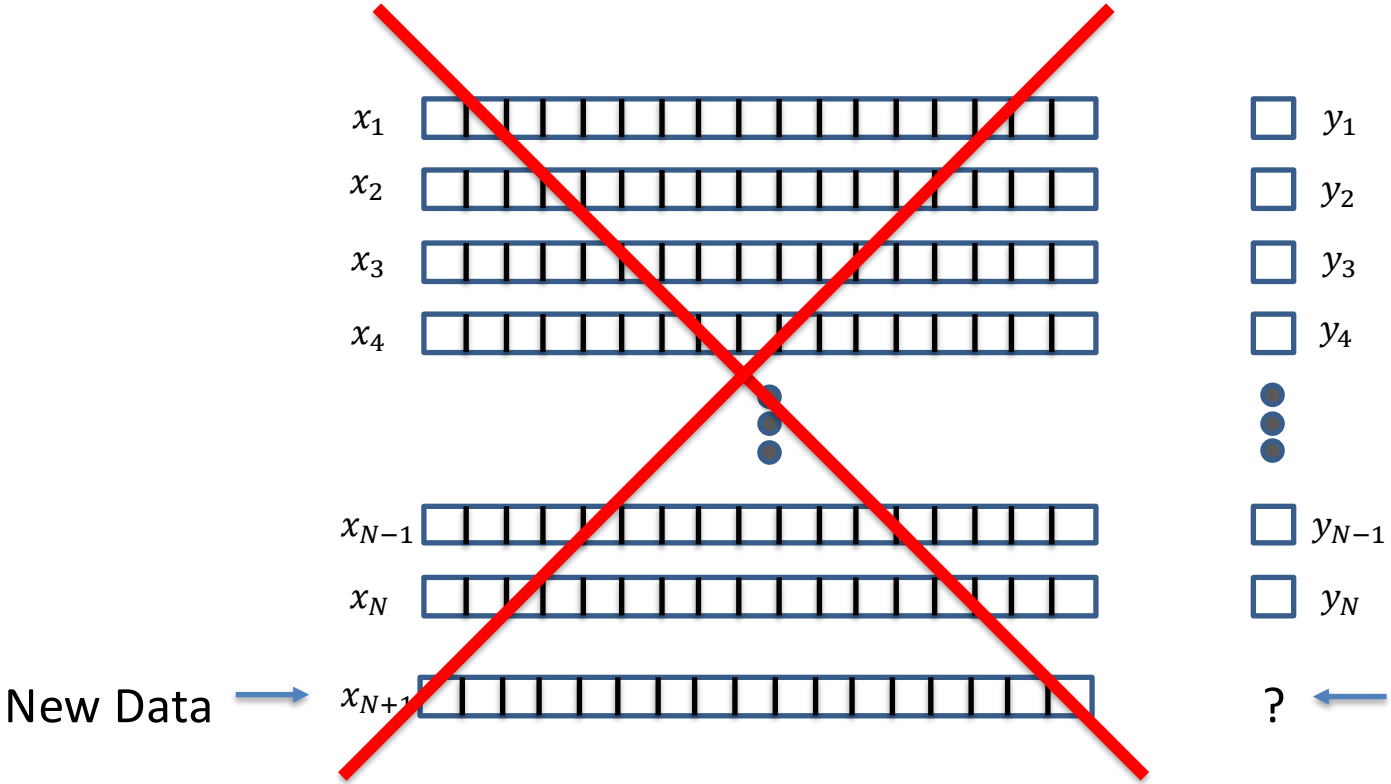


Start by limiting y to a binary outcome:

- False/True
- 0/1

Want to learn how to *predict* outcome

Our Previous Training Set Setup



Start by limiting y to a binary outcome:

- False/True
- 0/1

Want to learn how to *predict* outcome

Our Training Set is Text

x_1	"That movie was terrible!"	y_1	
x_2	"This is the best jacket…"	y_2	
x_3	"Most awful thing ever…"	y_3	Start by limiting y to a binary outcome:
x_4	"One of the most amazing…"	y_4	• False/True • 0/1
	⋮	⋮	
x_{N-1}	"Will never buy again…"	y_{N-1}	
x_N	"I'm rejuvenated…"	y_N	
New Data → x_{N+1}	"This class is great!"	? ←	Want to learn how to *predict* outcome

Algorithms work on numbers

- A simple question:

“Can we convert our text to a vector or sequence of numbers?”

- If yes, we can start using our previous methodology!

First approach: Counting

- Let us define a dictionary of vocabulary words
  - Each word is assigned an index
- Count the number of times that each word appears in each text example

"I am learning about deep learning"

↓ Counting word occurrences

learning:2,
I:1,
am:1,
about:1,
deep:1

Counting the number of occurrences

Original documents

perspective identifying tumor suppressor genes in human...
letters global warming report leslie roberts article global....
research news a small revolution gets under way the 1990s....
a continuing series the reign of trial and error draws to a close...
making deep earthquakes in the laboratory lab experimenters...
quick fix for freeways thanks to a team of fast working...
feathers fly in grouse population dispute researchers...

...

↓

Word index and counts

1897:1 1467:1 1351:1 731:2 800:5 682:1 315:6 3668:1 14:1
4261:2 518:1 271:6 2734:1 2662:1 2432:1 683:2 1631:7
2724:1 107:3 518:1 141:3 3208:1 32:1 2444:1 182:1 250:1
2552:1 1993:1 116:1 539:1 1630:1 855:1 1422:1 182:3 2432:1
1372:1 1351:1 261:1 501:1 1938:1 32:1 14:1 4067:1 98:2
4384:1 1339:1 32:1 4107:1 2300:1 229:1 529:1 521:1 2231:1
569:1 3617:1 3781:2 14:1 98:1 3596:1 3037:1 1482:12 665:2

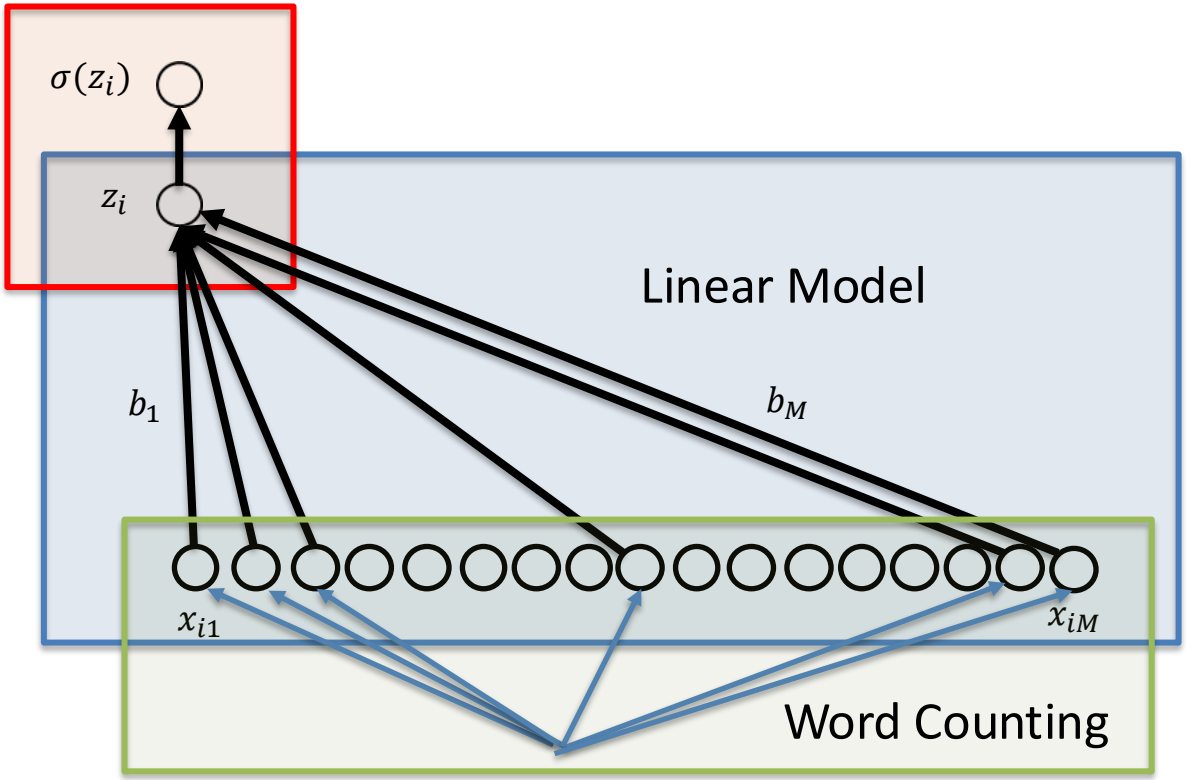
...

The format is: index:count

Each section of text is now a vector, where the mth entry is the number of times the mth dictionary word occurred in that example.

Revisiting Logistic Regression

Convert to Probability



"This class is great!"

Comments

- This word counting strategy can work decently well in practice
- Can easily put in our deep models instead
- But we are not really processing “natural” language
  - Does not capture word relationships (*e.g.*, “not bad”=“good”)
  - Does not capture word similarity (“cat” and “dog” are both pets, “lawyer” and “attorney” are synonyms)

Need to Understand Sequences

I have a… red / blue / green / black / white …car.

Let me talk to my… lawyer / attorney .

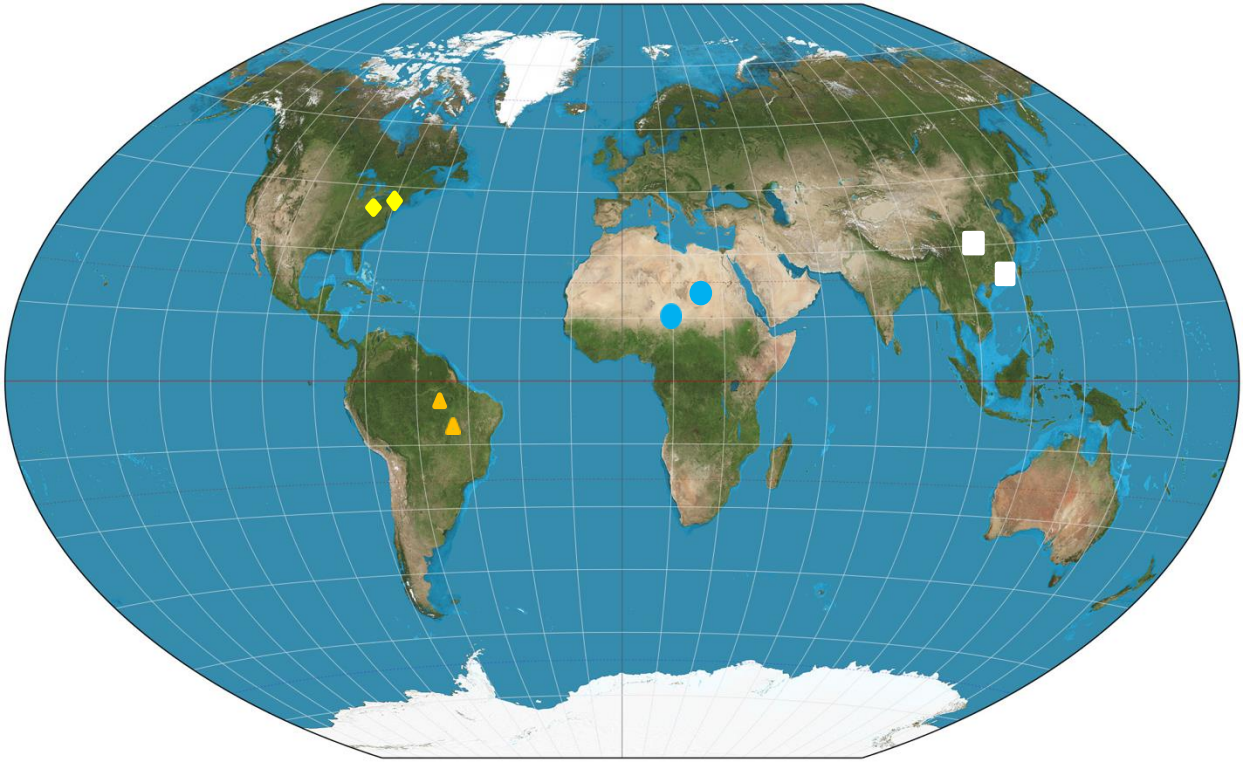
All are different sequences with different words, but we want to utilize the similarity.

Big Question:

- How can we represent the similarities between words?
- Can imagine using a thesaurus to define similarities, but that will be limited
- Need a (learnable) mathematical representation of word similarity

WORD EMBEDDINGS

Similarities Based on Proximity



- Points that are nearby spatially are likely to have similar attributes
- May think of each location is a point in a 2D space (lat, long)
- Measure similarity between places by their distance in 2D space

Map Each Word in Vocabulary to a Point in Space

- Each word in a vocabulary is mapped to a point in a 2D space
- The closer words are in the mapping, the more related/synonymous they are
- The algorithm will learn the 2D point at which to place each word

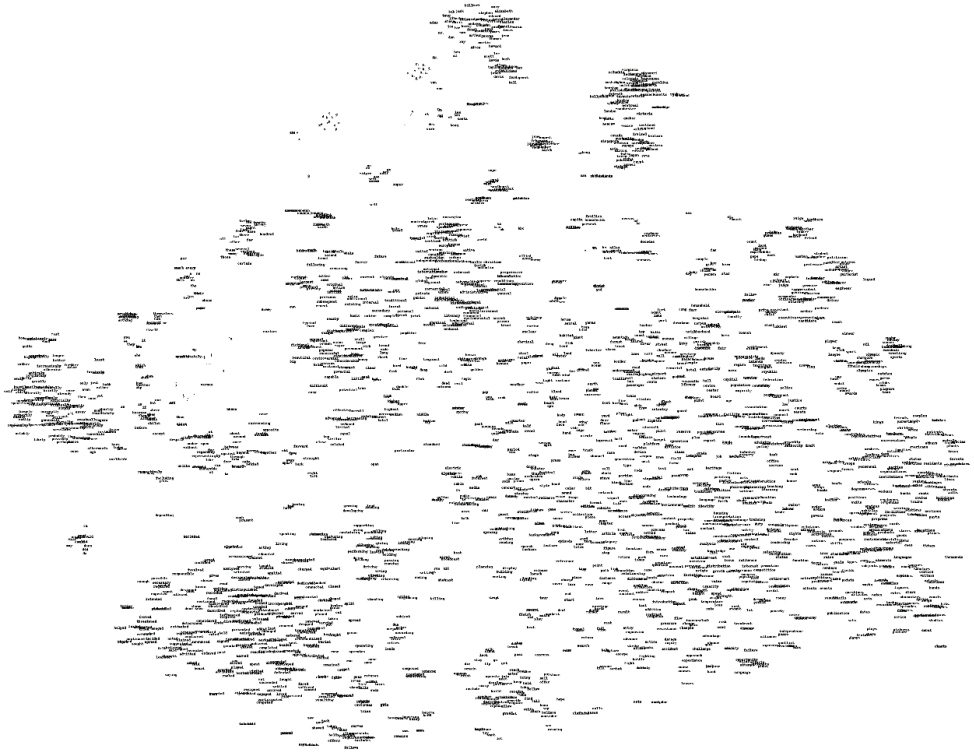
	lawyer 1	attorney 2	penguin 3	apple 4	●	●	●	V
longitude	78.8986	79.0558	135.0000	74.0060				
latitude	35.9940	35.9132	82.8628	40.7128				

vocabulary words

Example Word Geography

Here we show the learned geography of many different vocabulary words.

Too many words here to see! Let's zoom in on a smaller section.

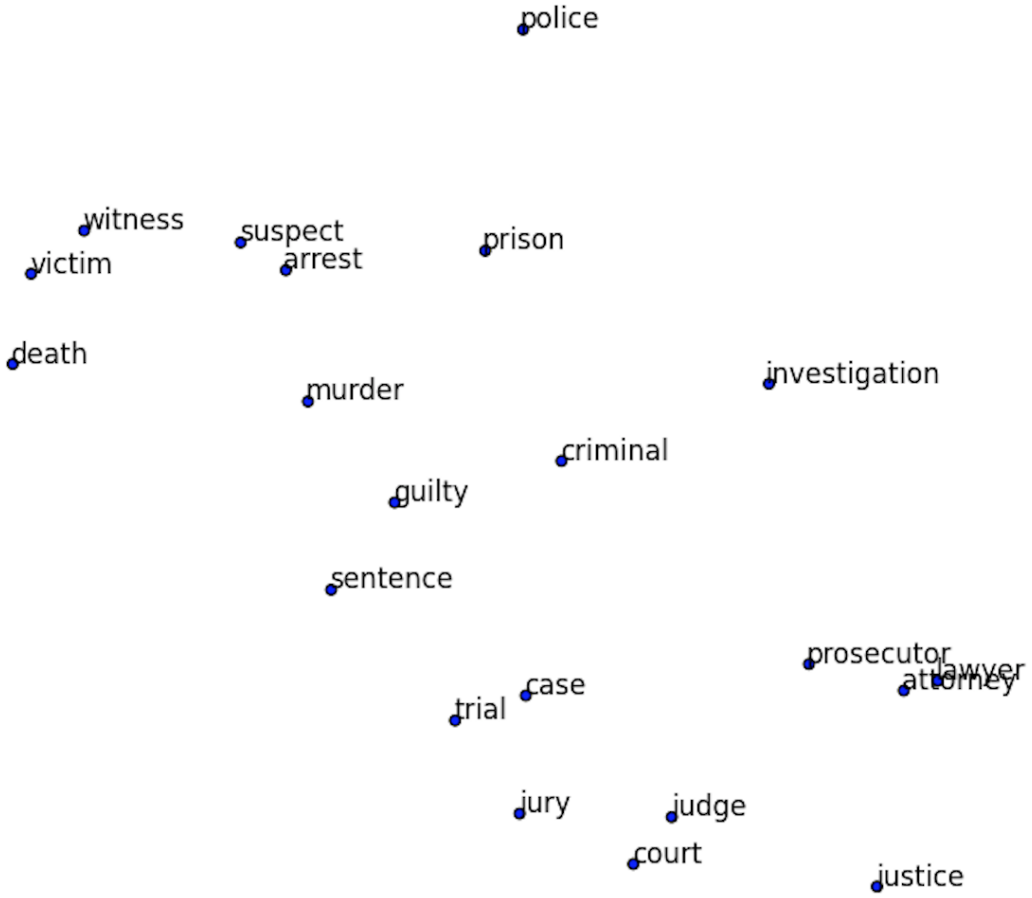


Example Word Geography

If we zoom in on a small region of our word map, it's all related words.

Note the similarity of all the words as a whole, but also of the individual neighbors.

"Lawyer" and "attorney" are nearly identical in space!



Word to Vector (Word2Vec)

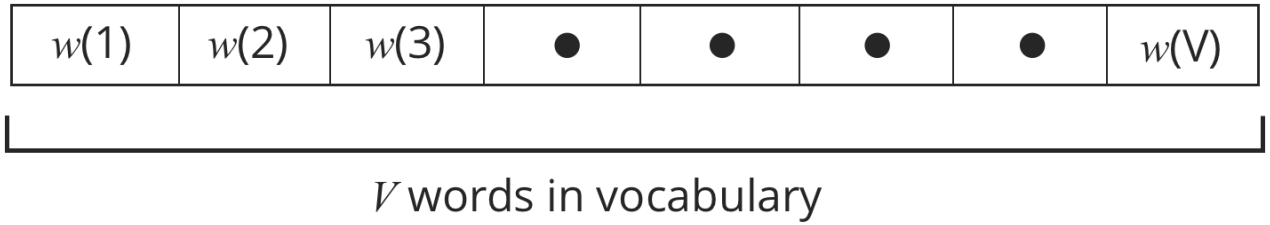
- The key to neural language models involves mapping each word to a vector
- Such word vectors are also called word "embeddings"
- Let $w(i)$ represent the i-th word in the vocabulary

	lawyer 1	attorney 2	penguin 3	apple 4	•	•	•	V
longitude	78.8986	79.0558	135.0000	74.0060				
latitude	35.9940	35.9132	82.8628	40.7128				

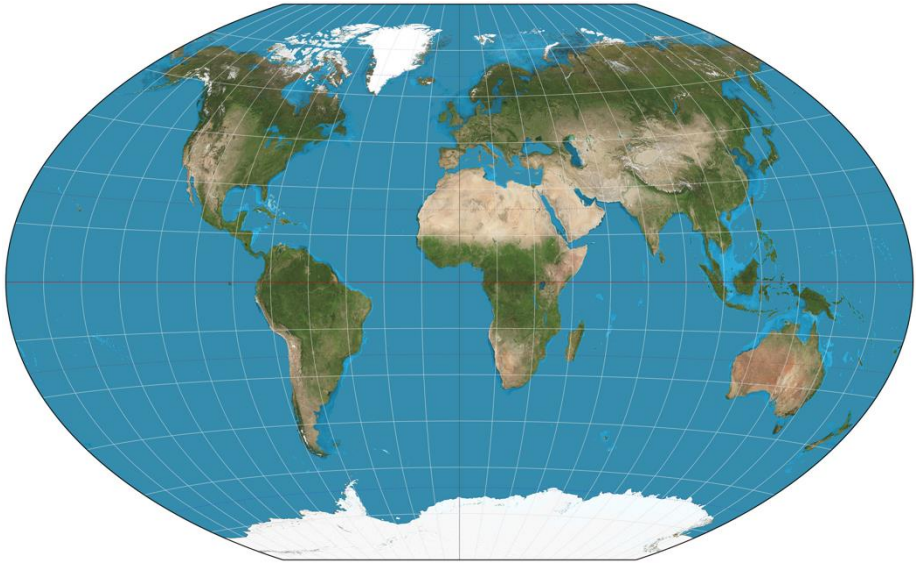
vocabulary words

Word to Vector (Word2Vec)

- Can think Word2Vec as a dictionary
  - Look up each word in the dictionary
  - If the word is in the dictionary, then we can look up the "definition" or location for it



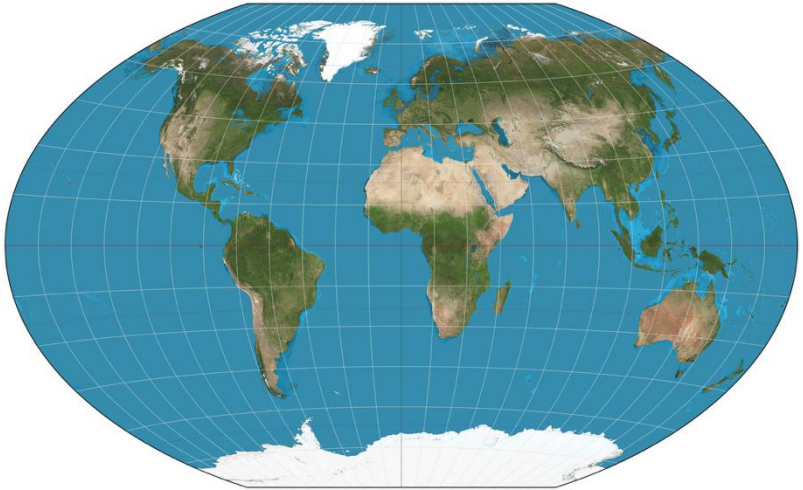
Do we use 2-dimensional embeddings?



	lawyer 1	attorney 2	penguin 3	apple 4	●	●	●	V
longitude	78.8986	79.0558	135.0000	74.0060				
latitude	35.9940	35.9132	82.8628	40.7128				

vocabulary words

Do we use 2-dimensional embeddings?



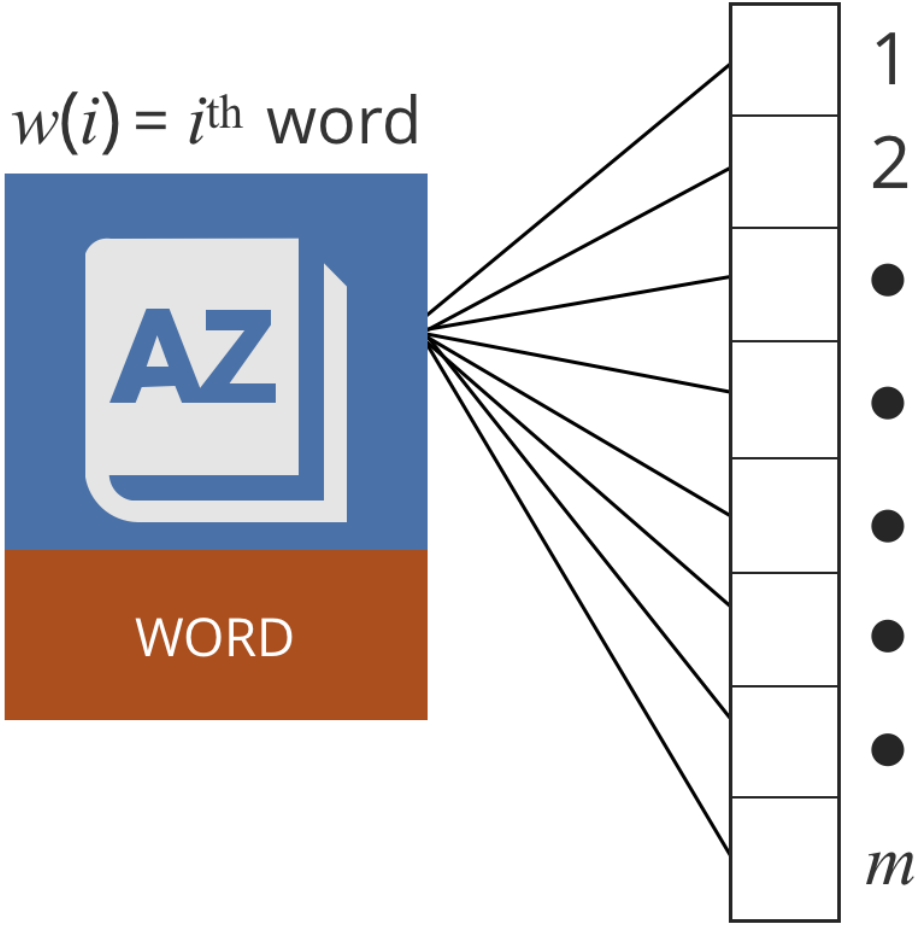
	lawyer 1	attorney 2	penguin 3	apple 4	●	●	●	V
longitude	78.8986	79.0558	135.0000	74.0060				
latitude	35.9940	35.9132	82.8628	40.7128				

vocabulary words

- 2-dimensional locations are useful to illustrate similarity
- *Computationally,* it turns out to be much easier to give each word a much longer address
  - Also, representation is easier:
    - 1 direction for plural
    - 1 direction for past tense
    - 1 direction for gender
    - *etc*.
- The location vector will have *M* different entries
- This is just like specifying a longer address (*e.g.*, state, city, zip, street, house color, car color, *etc*.)

Word to Vector (Word2Vec)

- The key to neural language models involves mapping each word to a vector
- Such word vectors are also called word "embeddings"
- Let $w(i)$ represent the ith word in the vocabulary



Conceptualize Meaning of Word Vectors

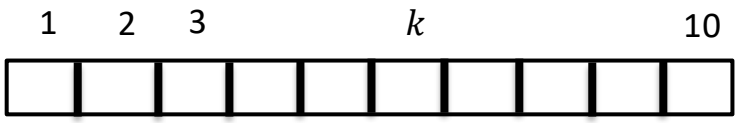
- Each component of the word vector may be viewed as being associated with a “topic”

Word represented by
10 components

Conceptualization of Meaning of Word Vectors

- Each component of the word vector may be viewed as being associated with a "topic"

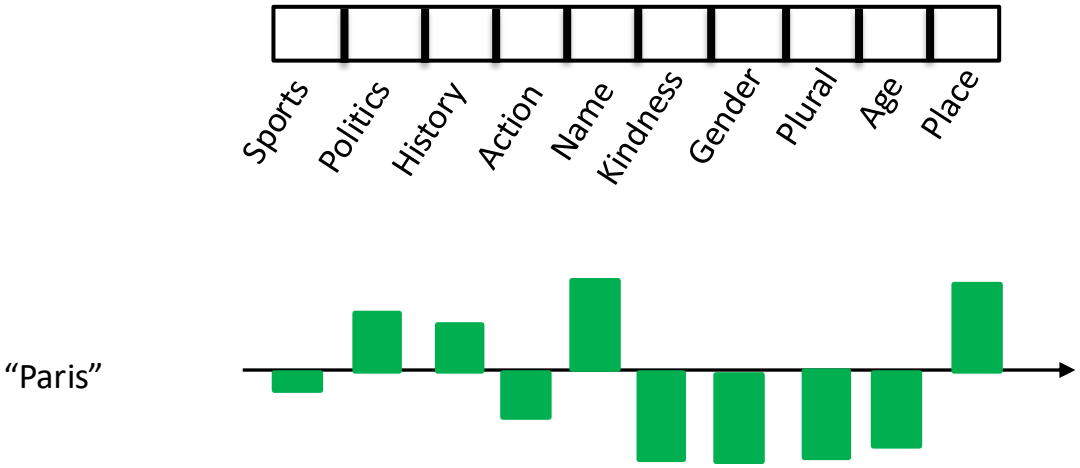
Word represented by
10 components



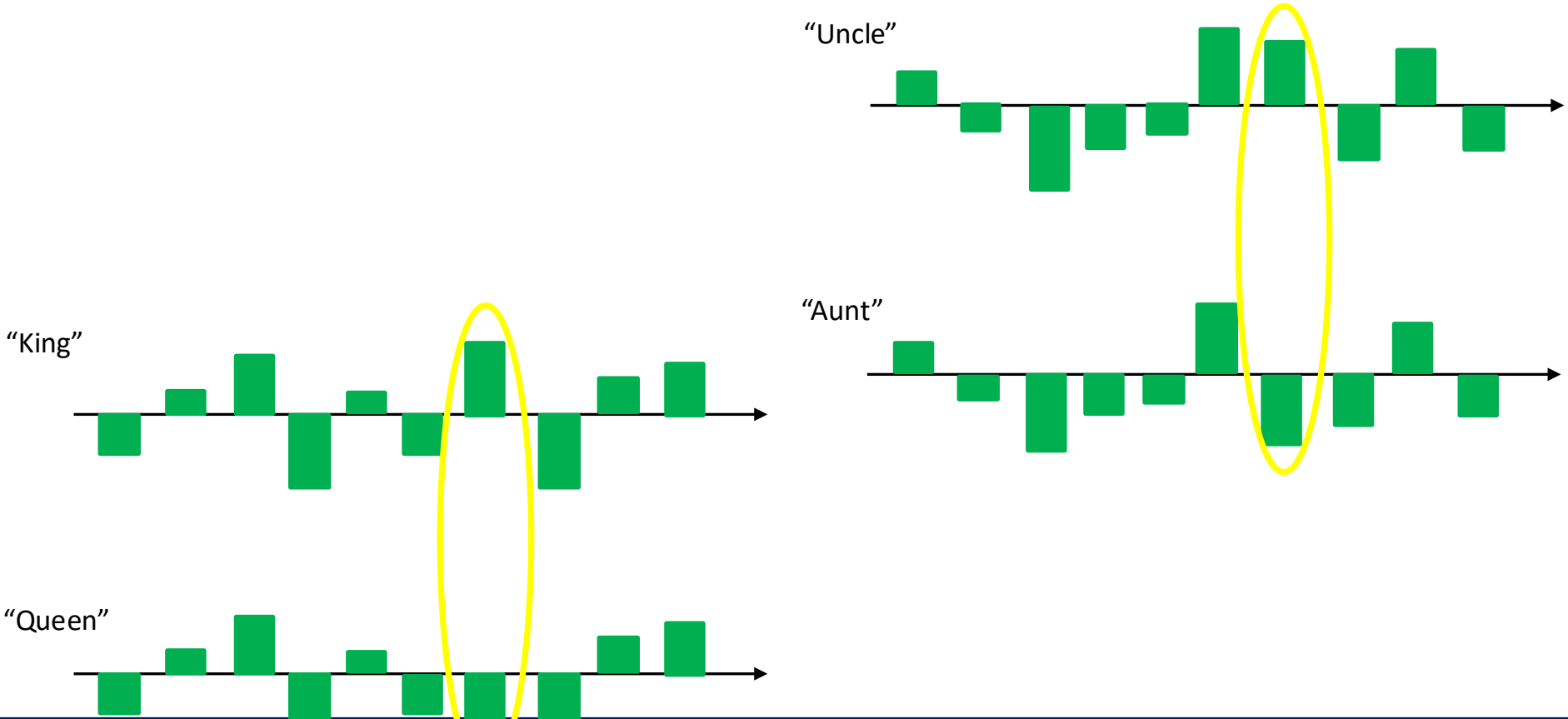
- Component k is positive if word aligned with Topic k, and component k is negative if word not aligned with Topic k

Notional Example

- Consider meanings for the 10 topics associated with the word vectors

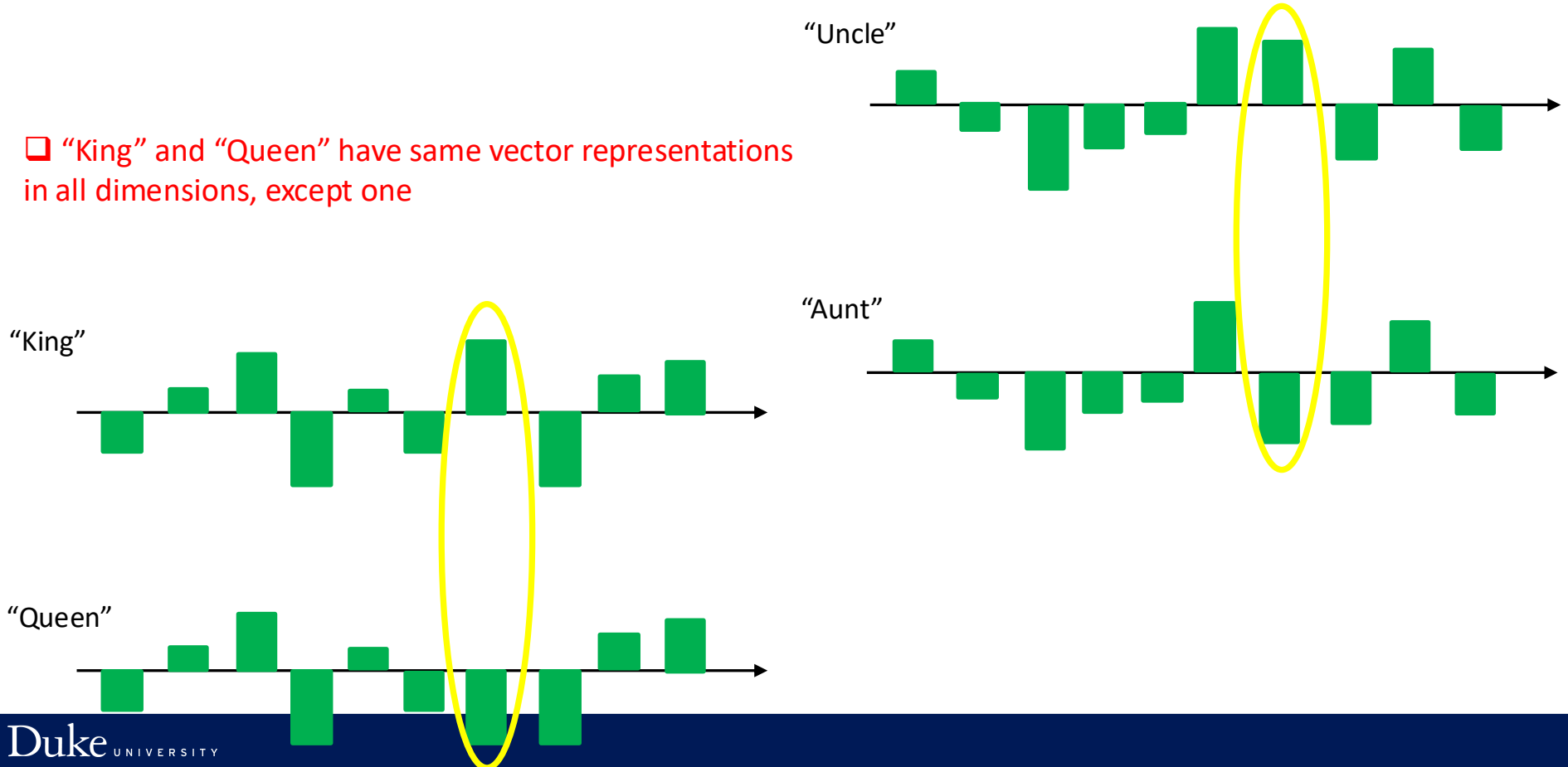


Preserve Relative Relationships Between Words

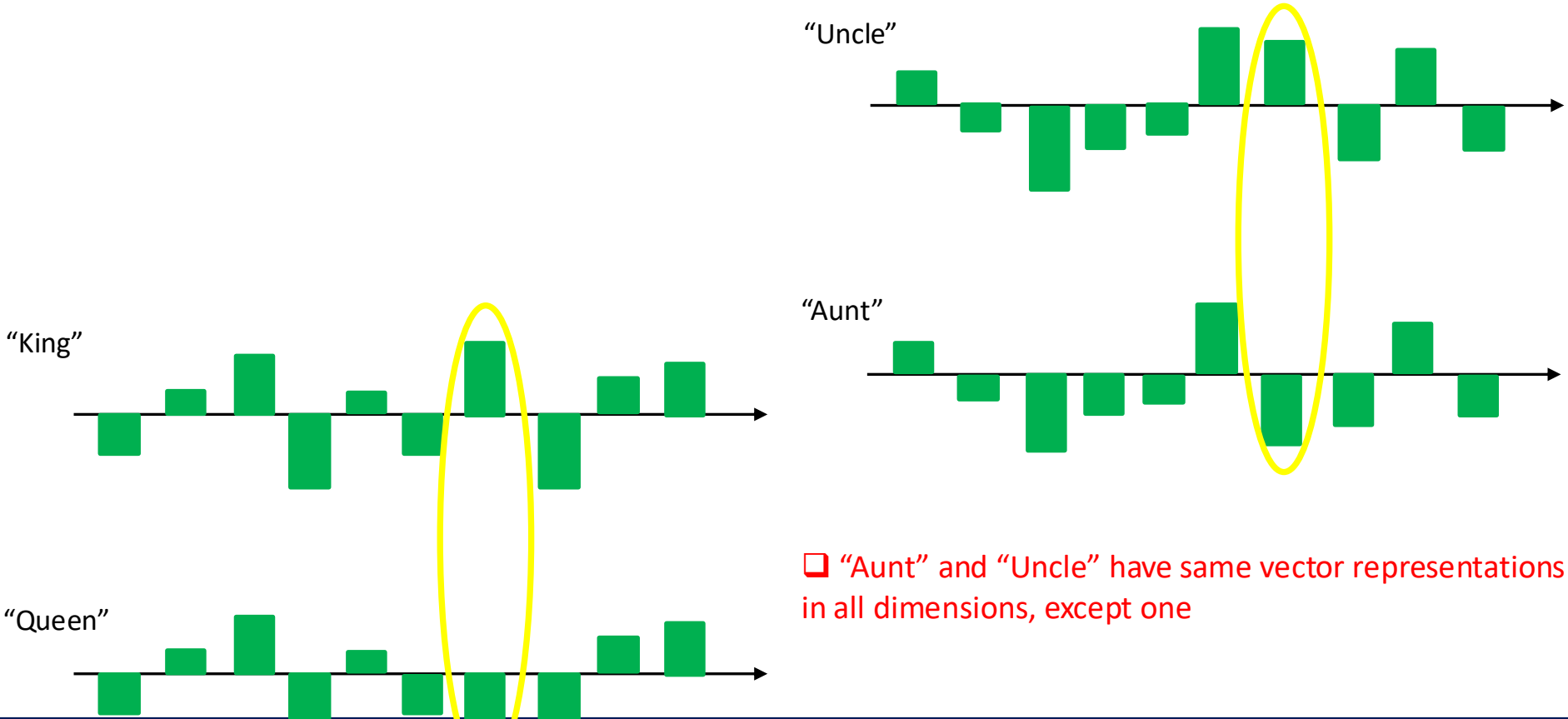


Preserve Relative Relationships Between Words

- "King" and "Queen" have same vector representations in all dimensions, except one

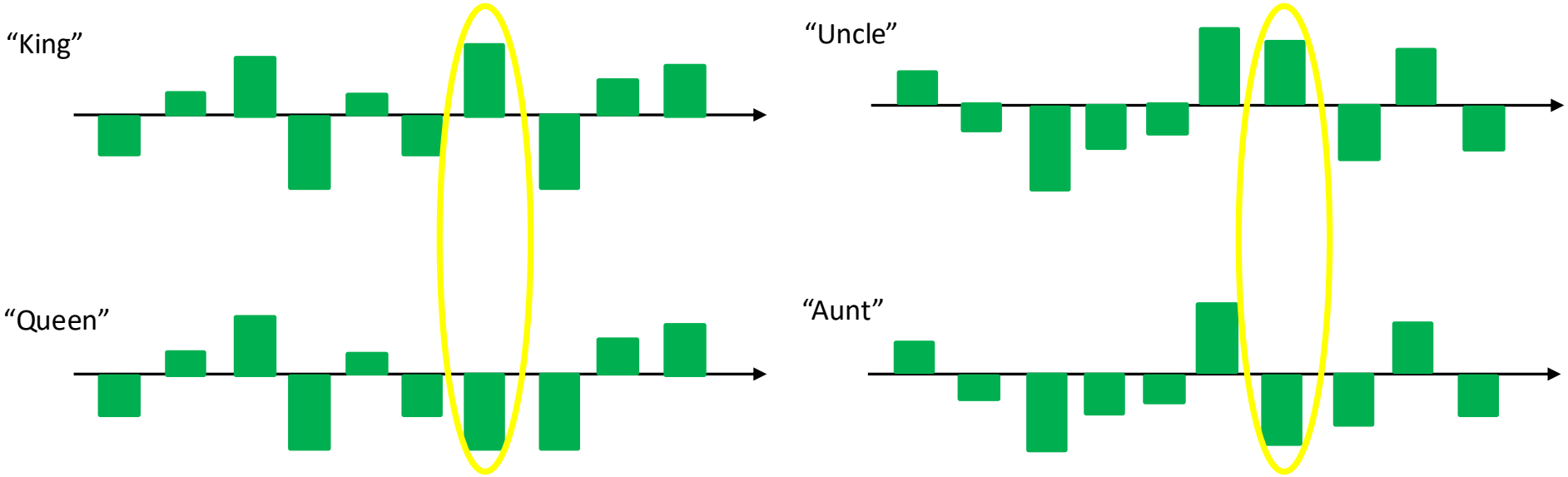


Preserve Relative Relationships Between Words



- "Aunt" and "Uncle" have same vector representations in all dimensions, except one

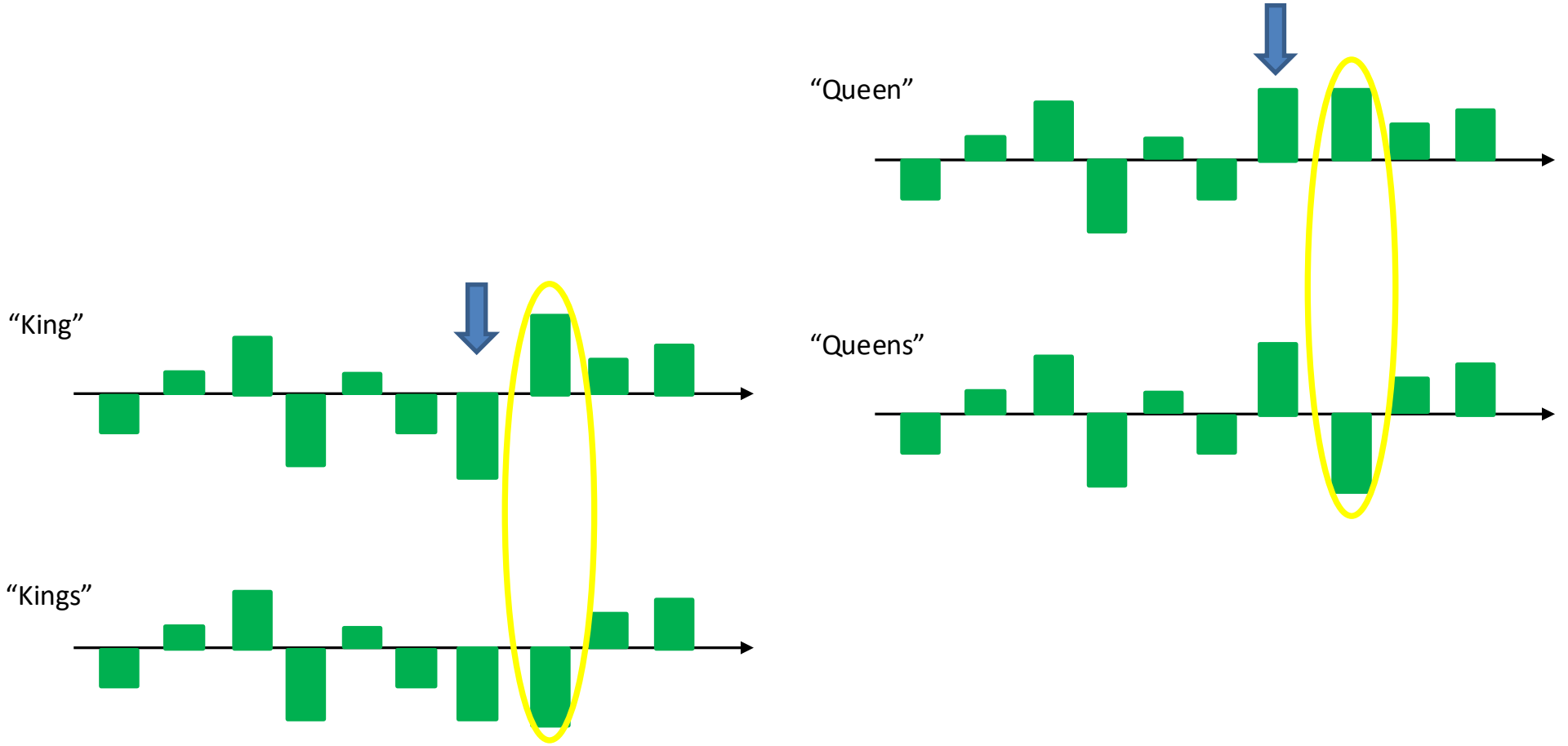
Preserve Relative Relationships Between Words



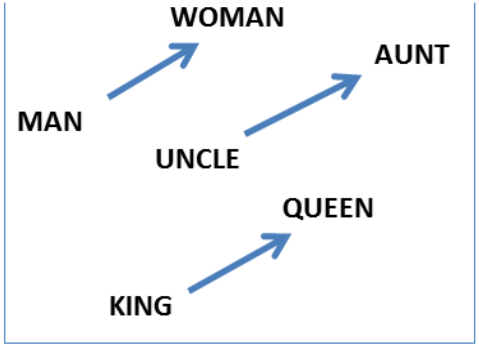
- ❑ Component of difference is the same in both pairs

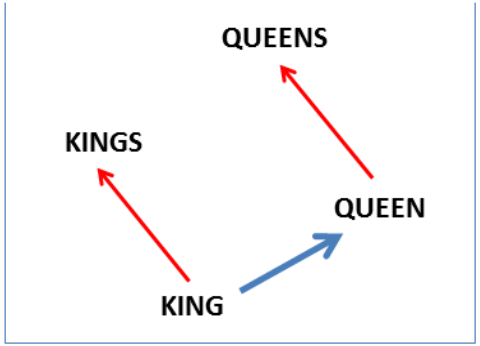
- ❑ Mechanism by which word vectors capture consistent relative relationships between words

Preserve Relative Relationships Between Words



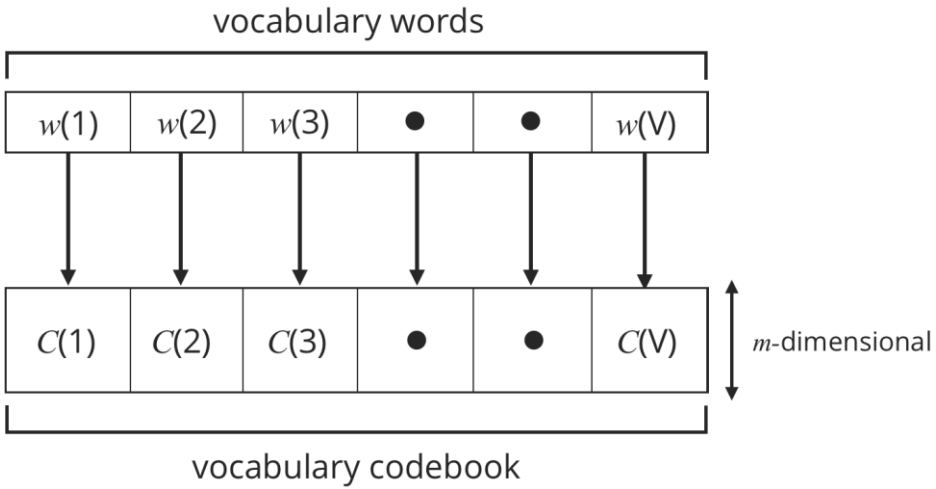
Preserve Relative Relationships Between Words



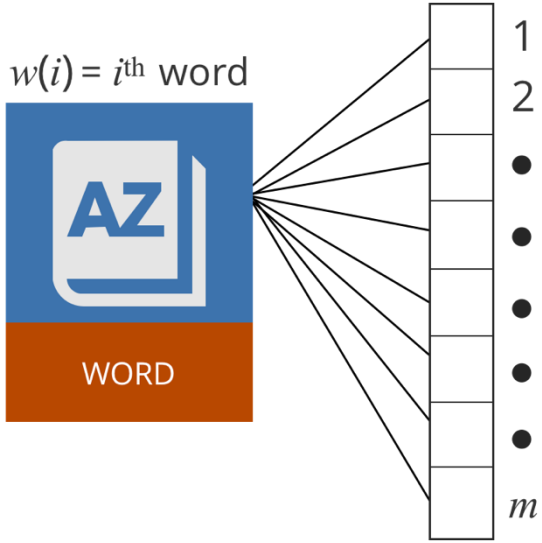


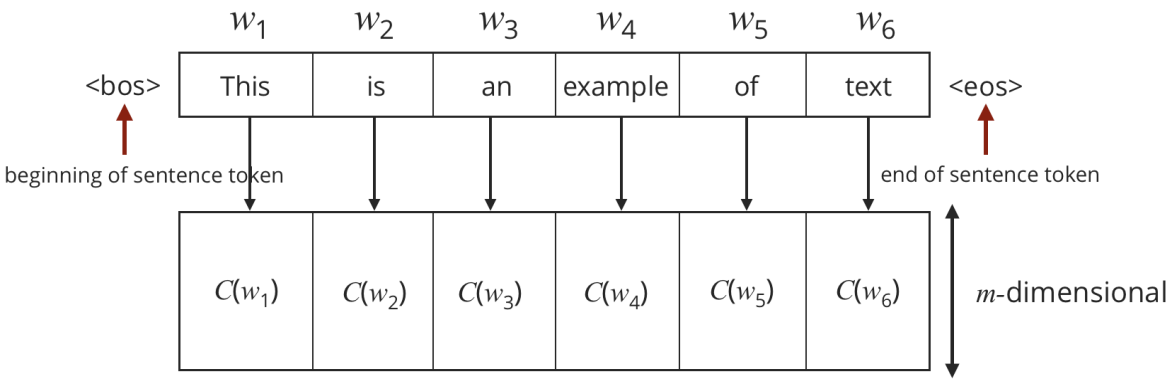
Word to Vector (Word2Vec)

- The set of words in the vocabulary mapped to a codebook of vectors



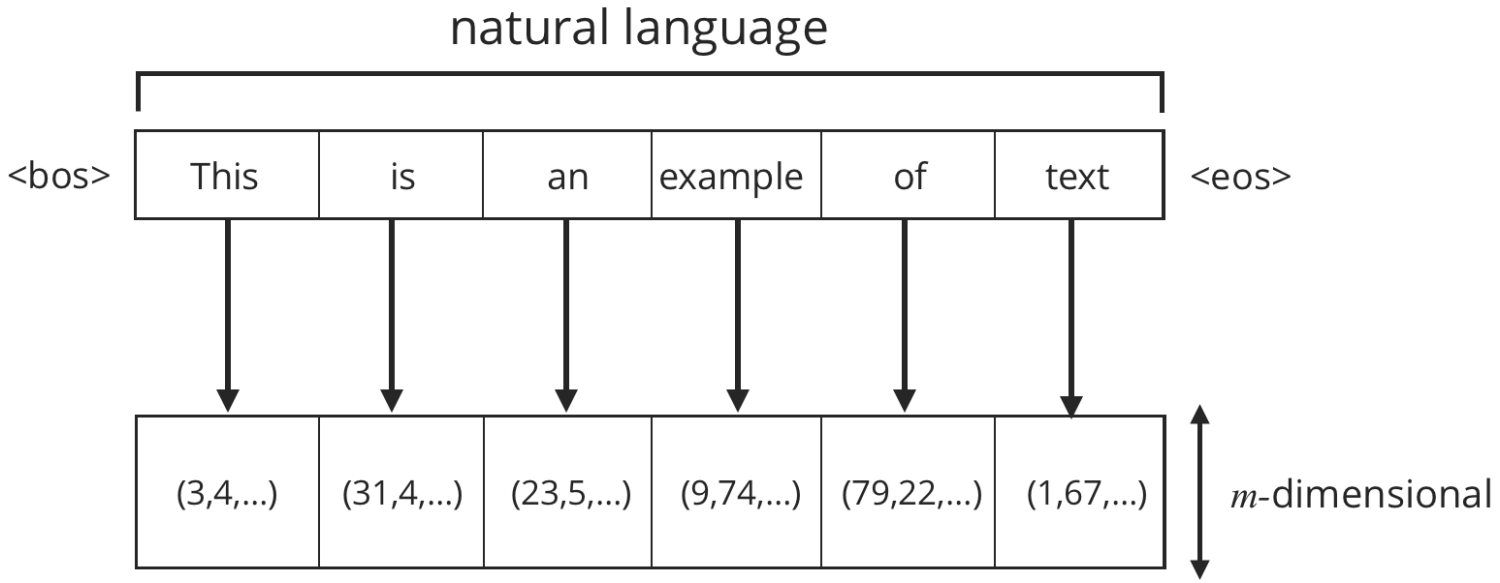
Looking up locations in a dictionary



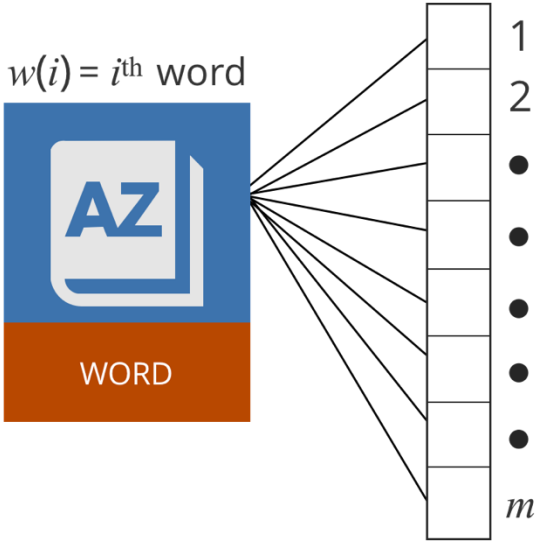


Converting a sentence to embeddings

- We can construct a sequence of word embeddings to represent each sentence.
  - Each word is looked up in our dictionary individually
  - Specific tokens to mark “beginning of sentence” (<bos>) and “end of sentence” (<eos>)

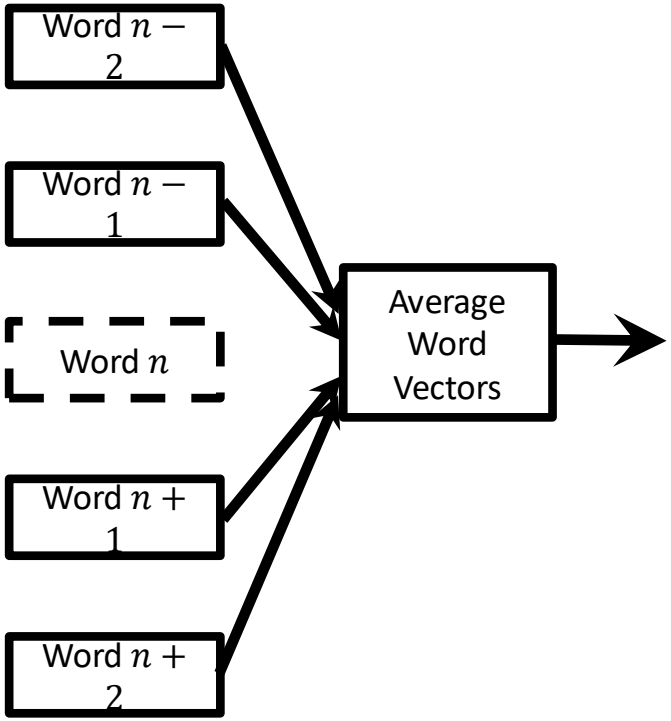


How can we learn the dictionary?

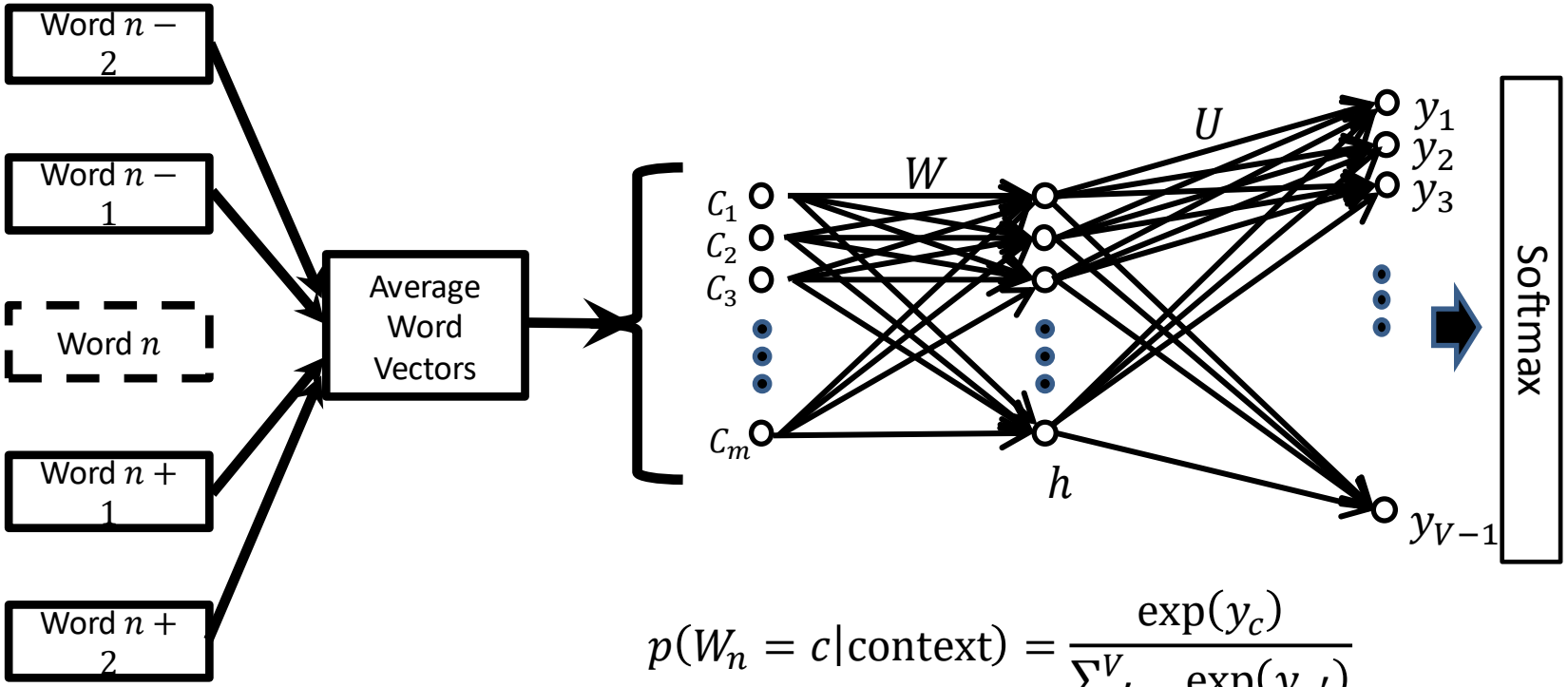


- We do not start knowing this dictionary—must be learned from the data

Continuous Bag of Words (CBOW) Model



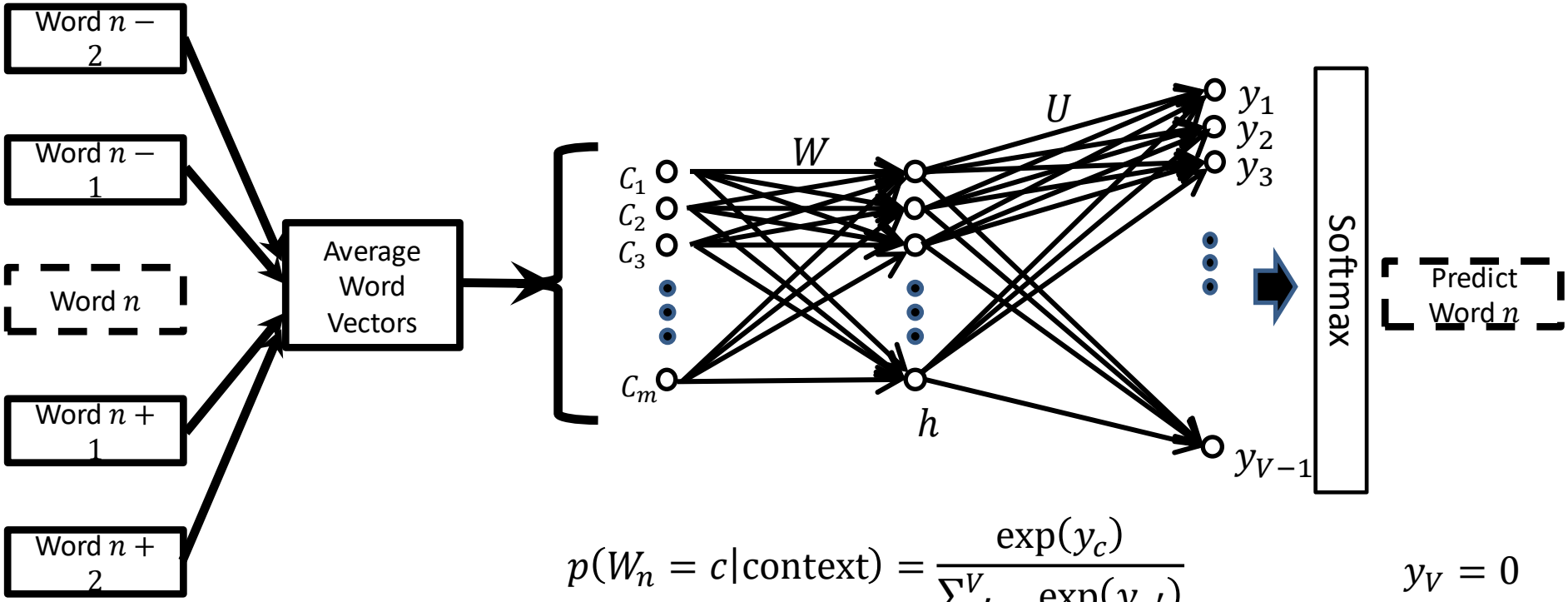
Continuous Bag of Words (CBOW) Model



$$p(W_n = c|\text{context}) = \frac{\exp(y_c)}{\sum_{c'=1}^{V} \exp(y_{c'})}$$

$$y_V = 0$$

Continuous Bag of Words (CBOW) Model

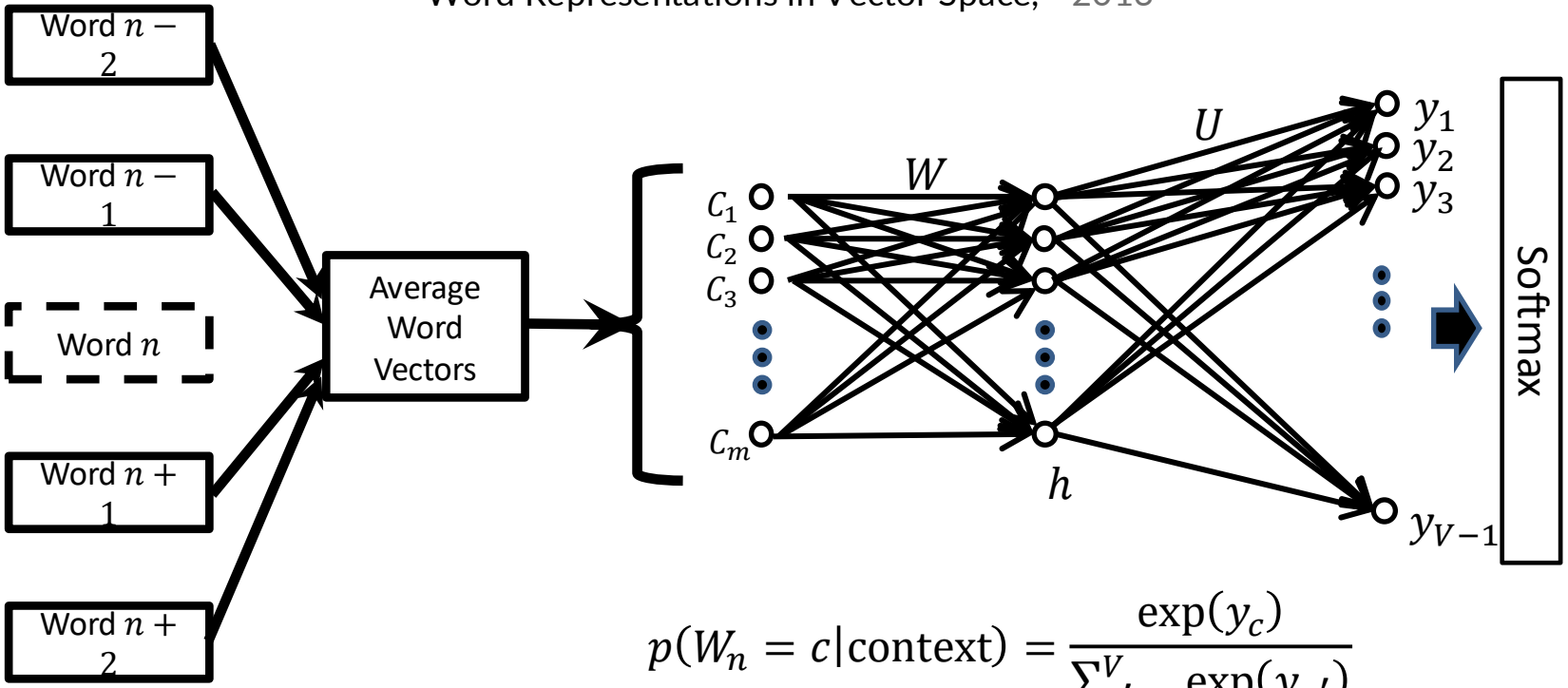


$$p(W_n = c|\text{context}) = \frac{\exp(y_c)}{\sum_{c'=1}^{V} \exp(y_{c'})}$$

$$y_V = 0$$

Continuous Bag of Words (CBOW) Model

T. Mikolov, K. Chen, G. Corrado and J. Dean, "Efficient Estimation of Word Representations in Vector Space," 2013

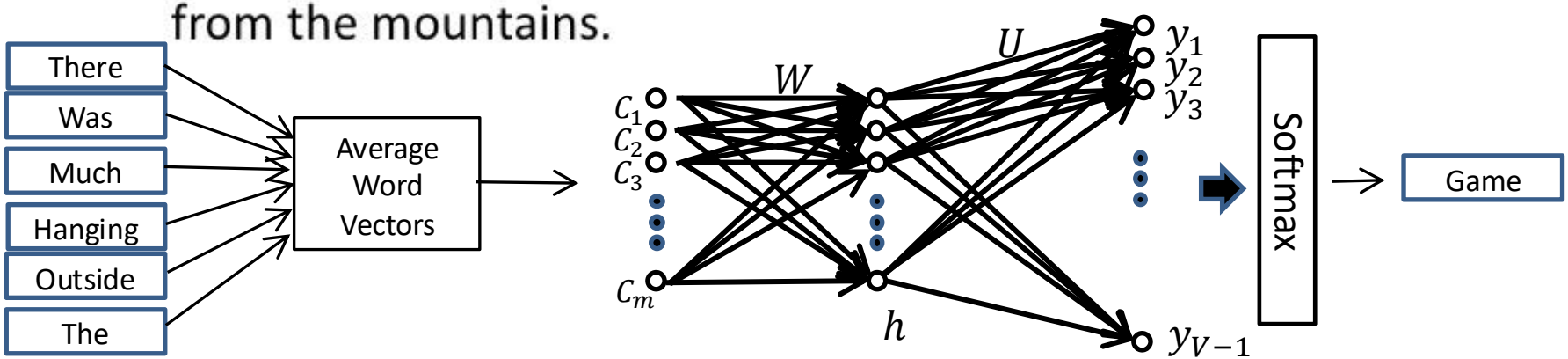


$$p(W_n = c|\text{context}) = \frac{\exp(y_c)}{\sum_{c'=1}^{V} \exp(y_{c'})}$$

$$y_V = 0$$

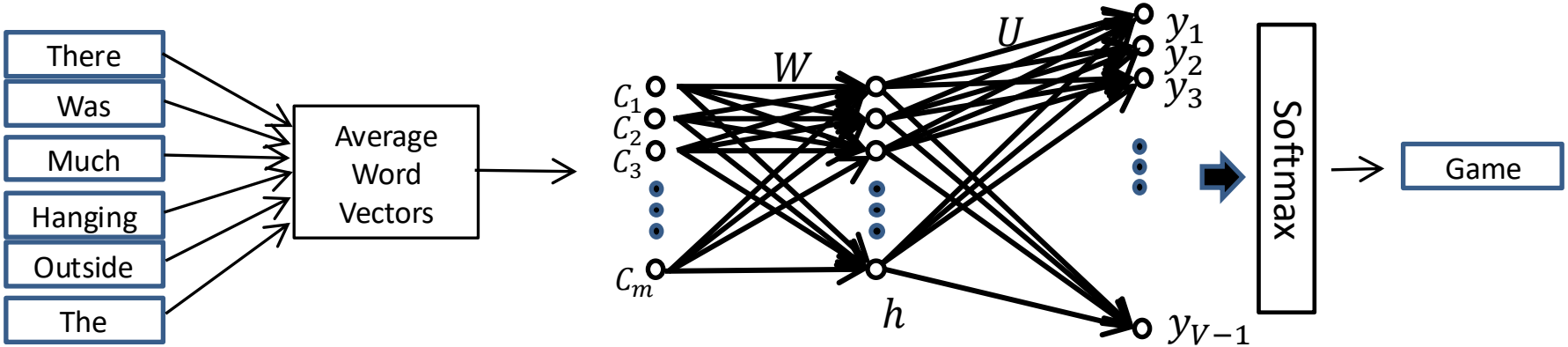
In the fall the war was always there, but we did not go to it any more. It was cold in the fall in Milan and the dark came very early. Then the electric lights came on, and it was pleasant along the streets looking in the windows. There was much game hanging outside the shops, and the snow powdered in the fur of the foxes and the wind blew their tails. The deer hung stiff and heavy and empty, and small birds blew in the wind and the wind turned their feathers. It was a cold fall and the wind came down from the mountains.

In the fall the war was always there, but we did not go to it any more. It was cold in the fall in Milan and the dark came very early. Then the electric lights came on, and it was pleasant along the streets looking in the windows. There was much game hanging outside the shops, and the snow powdered in the fur of the foxes and the wind blew their tails. The deer hung stiff and heavy and empty, and small birds blew in the wind and the wind turned their feathers. It was a cold fall and the wind came down from the mountains.



Learning with CBOW

- There is a true word and a prediction
- Learn the model parameters *C, W, U, b* to predict the true word well
- Uses same stochastic gradient descent techniques as before



Pros & Cons of CBOW

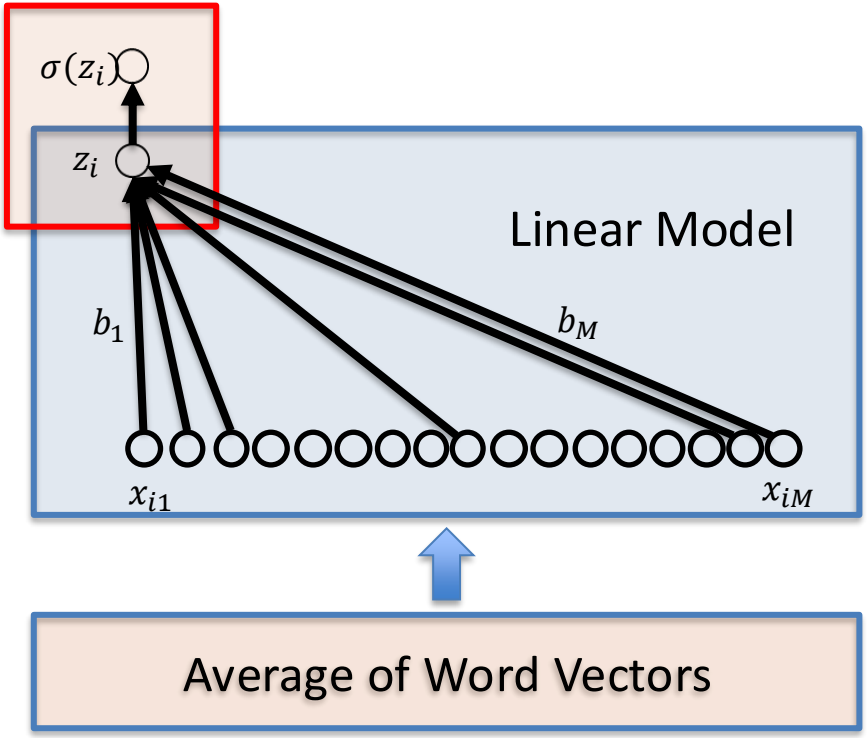
- Pro: Unsupervised, requires no labels
- Con: Assumes that word embeddings are independent of context
- Con: Does not account for word order in the sentence(s)
- With large language models, we will leverage the above pro, and address both of the cons
- For sentiment analysis we have labeled data, so let's use it
- Important: The word embeddings we learn for sentiment analysis will typically be different than those used for language modeling

Sentiment Analysis: Supervised Learning

x_1	"That movie was terrible!"	y_1	
x_2	"This is the best jacket…"	y_2	
x_3	"Most awful thing ever…"	y_3	Start by limiting y to a binary outcome:
x_4	"One of the most amazing…"	y_4	• False/True • 0/1
	⋮	⋮	
x_{N-1}	"Will never buy again…"	y_{N-1}	
x_N	"I'm rejuvenated…"	y_N	
New Data → x_{N+1}	"This class is great!"	?	← Want to learn how to *predict* outcome

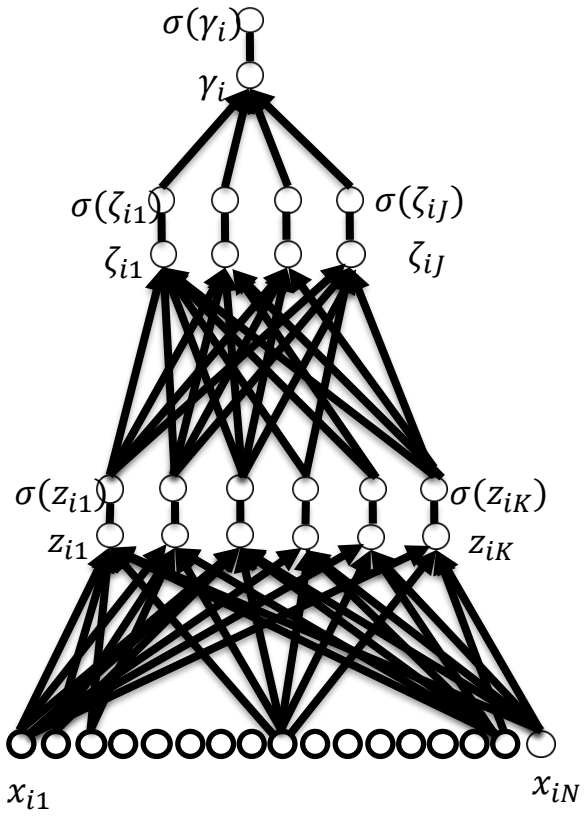
Logistic Regression for Sentiment Analysis

- Training example (text) i
- Have sentiment label $y_i \in \{0,1\}$
- Every word in text mapped to an M-dimensional vector (embedding)
- Average word embeddings and feed into logistic regression
- Cross-entropy loss
- Learn word vectors and logistic-regression parameters

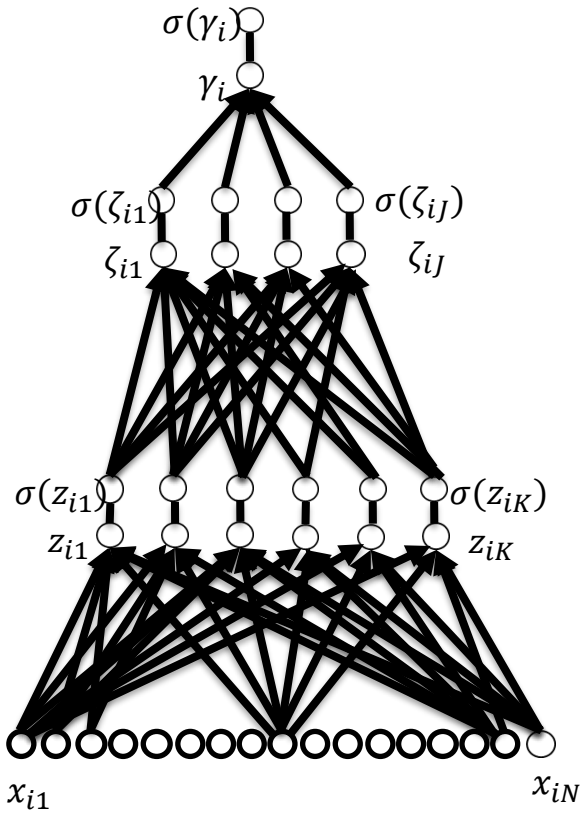


Performing Gradients Efficiently in Deep NNs

Leveraging NN Structure for Efficient Gradients



Leveraging NN Structure for Efficient Gradients

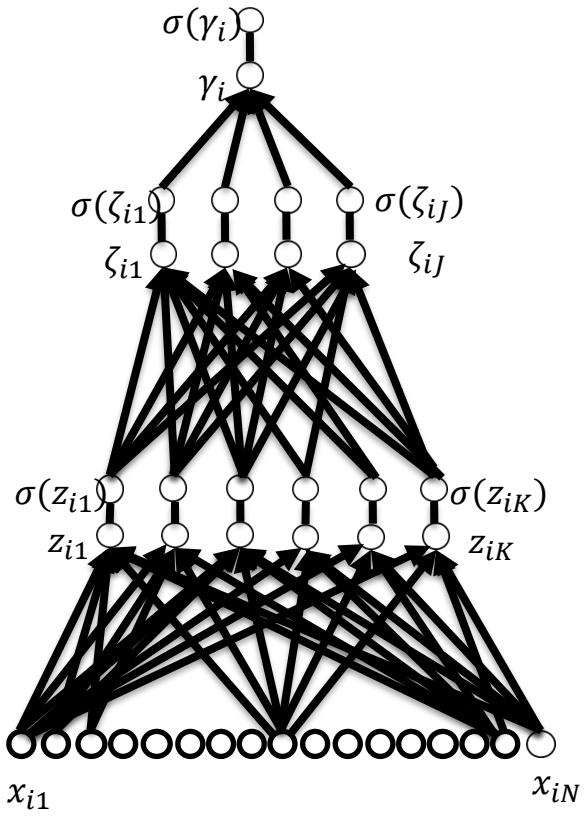


$h_3 \in R$

$h_2 \in R^J$

$h_1 \in R^K$

Leveraging NN Structure for Efficient Gradients



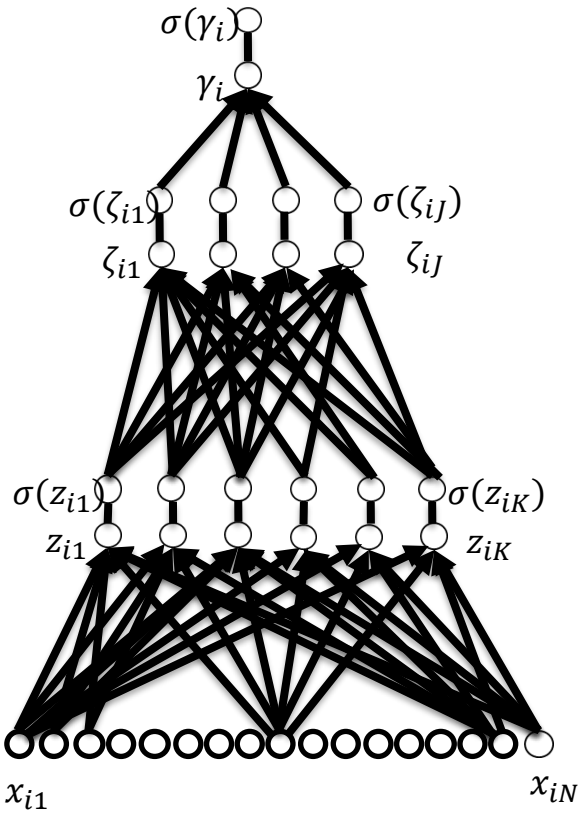
$h_3 \in R$

$h_2 \in R^J$

$h_1 \in R^K$

- "Hidden" vector at output of nonlinearity at each layer of deep network
- For current setting of the model parameters, perform a forward pass of computations through the model
- Store the hidden vector emitted from each layer of NN

Leveraging NN Structure for Efficient Gradients



$h_3 \in R$

$h_2 \in R^J$

$h_1 \in R^K$

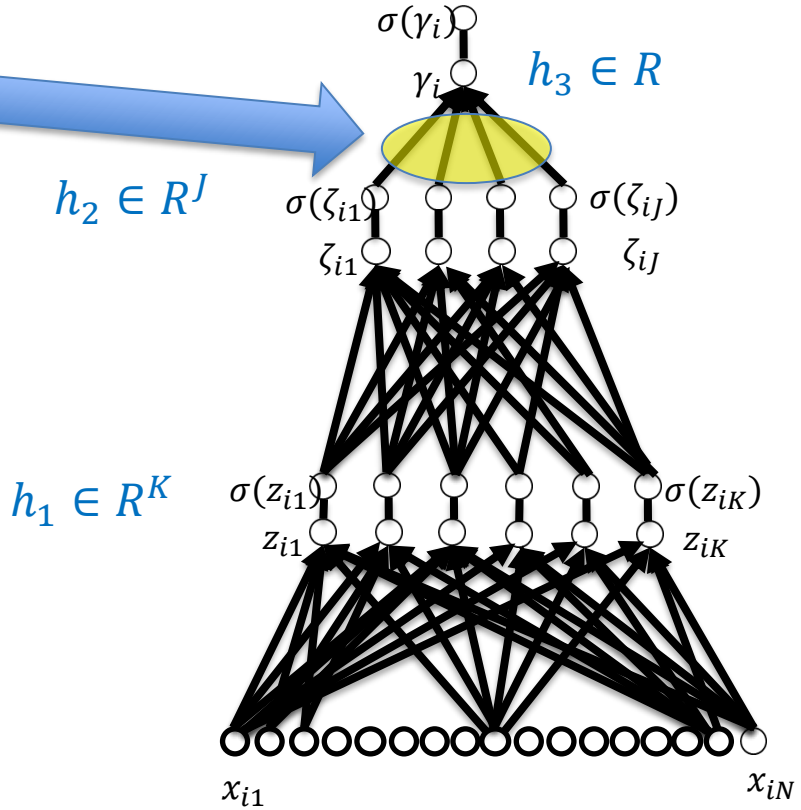
Forward pass

Backward Pass of Gradients, Using h_1, h_2 & h_3

Start by taking gradients of parameters nearest output

(w, b)

$$h_3 = \gamma_i = f[w, b, h_2(x_i)]$$



Backward Pass of Gradients, Using h_1, h_2 & h_3

Start by taking gradients of parameters nearest output

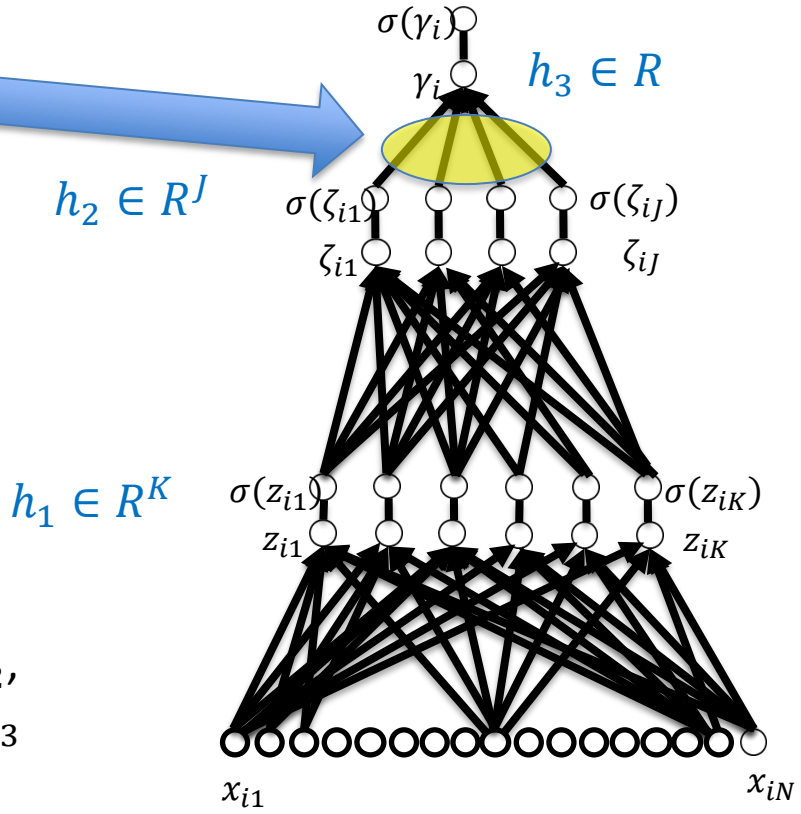
(w, b)

$$h_3 = \gamma_i = f[w, b, h_2(x_i)]$$

$$\frac{\partial \ell(x_i, y_i)}{db} = \frac{\partial h_3}{\partial b} \frac{\partial \ell(x_i, y_i)}{dh_3}$$

$$\frac{\partial \ell(x_i, y_i)}{dw_k} = \frac{\partial h_3}{\partial w_k} \frac{\partial \ell(x_i, y_i)}{dh_3}$$

Depends on h_2, evaluated at h_3



Backward Pass of Gradients, Using h_1, h_2 & h_3

Start by taking gradients of parameters nearest output

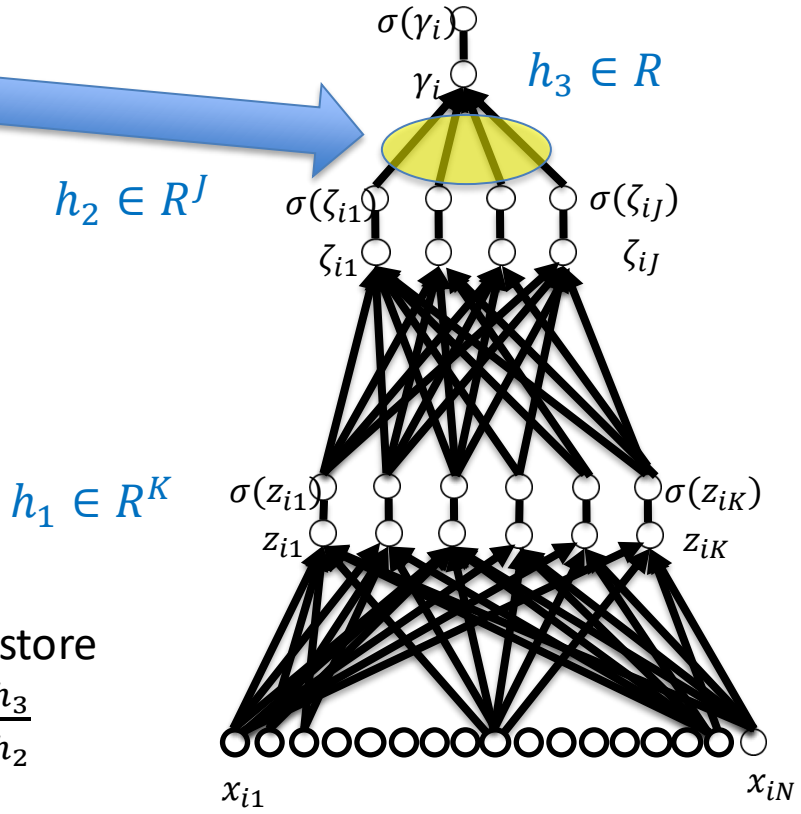
(w, b)

$$h_3 = \gamma_i = f[w, b, h_2(x_i)]$$

$$\frac{\partial \ell(x_i, y_i)}{db} = \frac{\partial h_3}{\partial b} \frac{\partial \ell(x_i, y_i)}{dh_3}$$

$$\frac{\partial \ell(x_i, y_i)}{dw_k} = \frac{\partial h_3}{\partial w_k} \frac{\partial \ell(x_i, y_i)}{dh_3}$$

After this step, store $\frac{\partial \ell(x_i, y_i)}{dh_3}$ & $\frac{\partial h_3}{\partial h_2}$

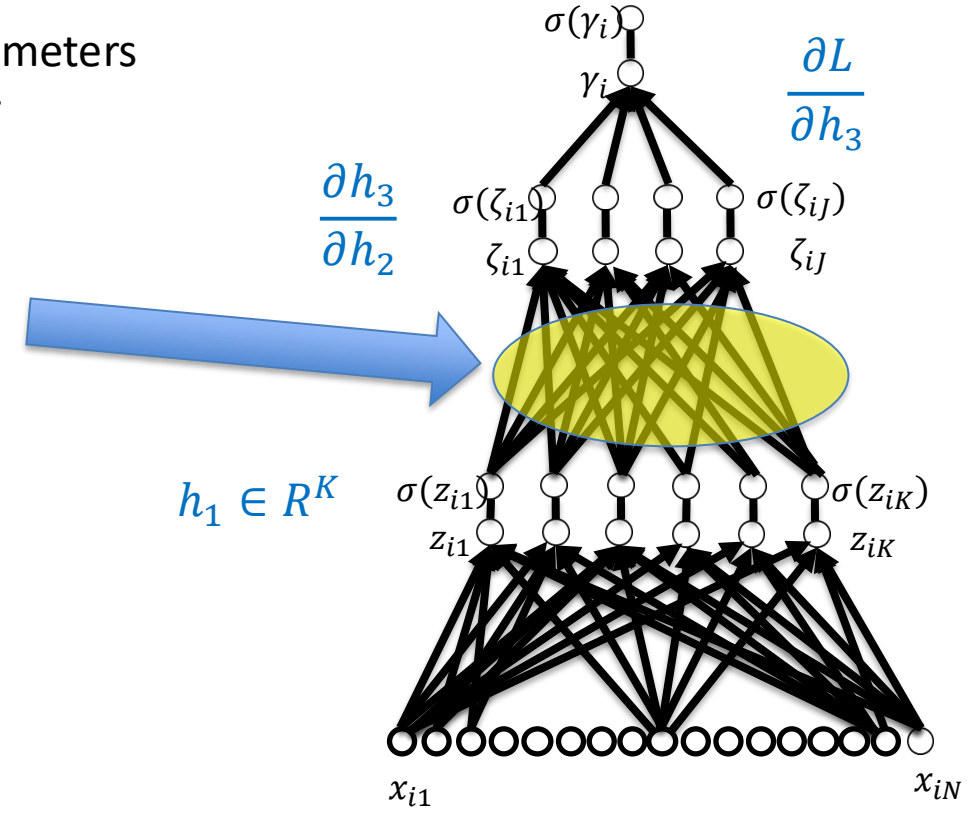


Backward Pass of Gradients, Using h_1, h_2 & h_3

Move backward to parameters
at the prior layer

(Z_2, b_2)

$h_2 = g[Z_2, b_2, h_1(x_i)]$



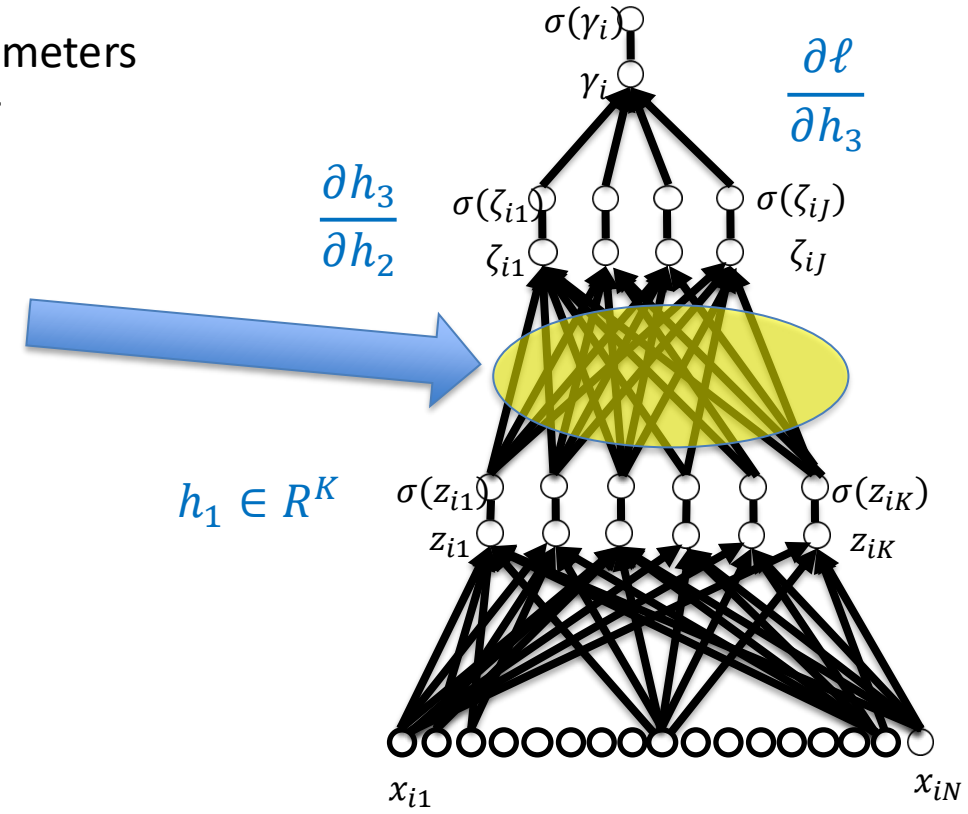
Backward Pass of Gradients, Using h_1, h_2 & h_3

Move backward to parameters
at the prior layer

$$(Z_2, b_2)$$

$$h_2 = g[Z_2, b_2, h_1(x_i)]$$

$$\frac{\partial \ell}{\partial Z_{2,jk}} = \frac{\partial \ell}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial Z_{2,jk}}$$



Backward Pass of Gradients, Using h_1, h_2 & h_3

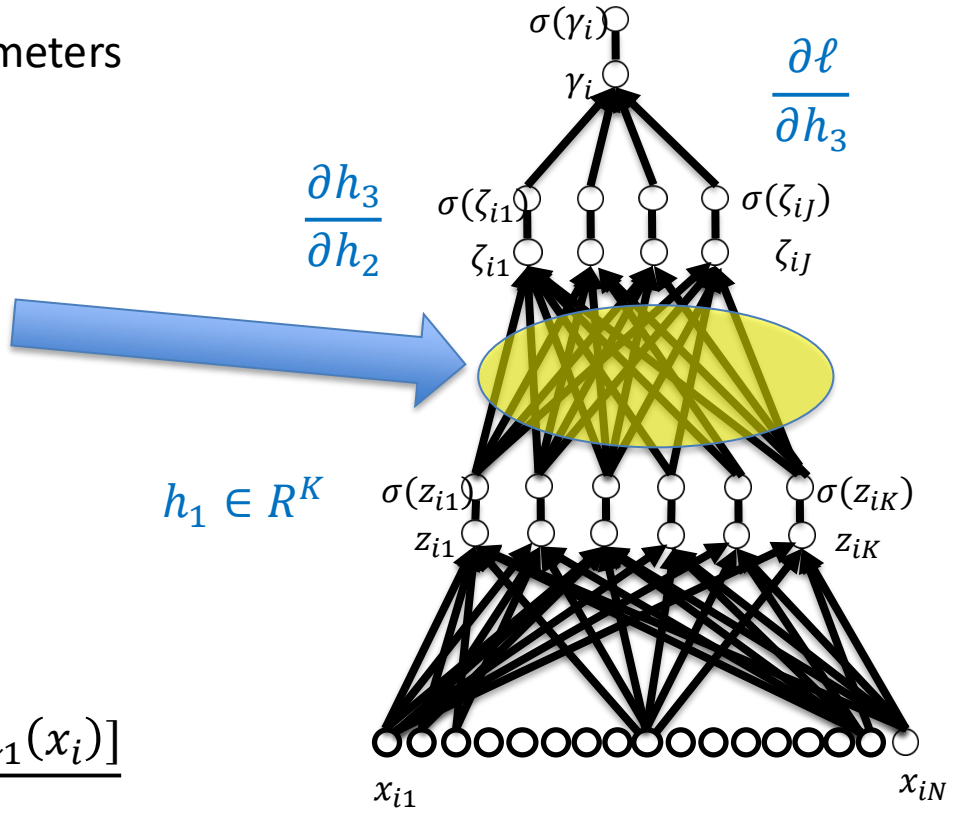
Move backward to parameters at the prior layer

(Z_2, b_2)

$$h_2 = g[Z_2, b_2, h_1(x_i)]$$

$$\frac{\partial \ell}{\partial Z_{2,jk}} = \frac{\partial \ell}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial Z_{2,jk}}$$

$$= \frac{\partial \ell}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial g[Z_2, b_2, h_1(x_i)]}{\partial Z_{2,jk}}$$

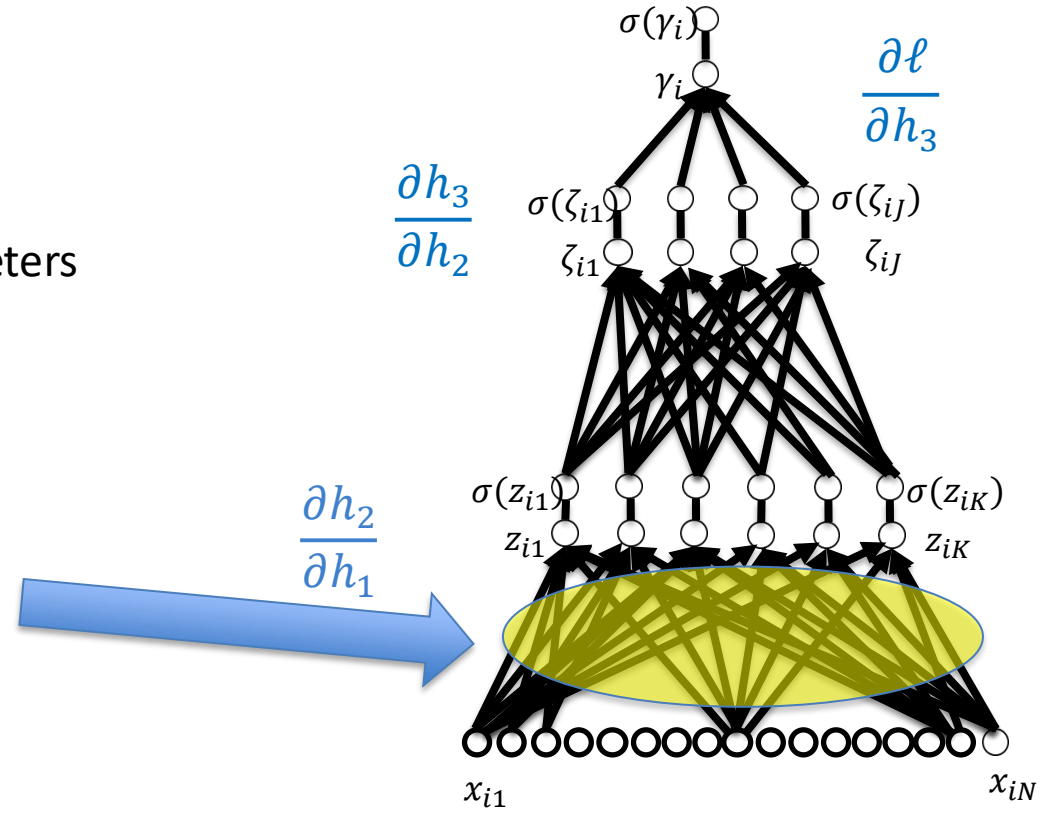


Backward Pass of Gradients, Using h_1, h_2 & h_3

Move backward to parameters at the prior layer

$$(Z_1, b_1)$$

$$h_1 = h[Z_1, b_1, x_i]$$



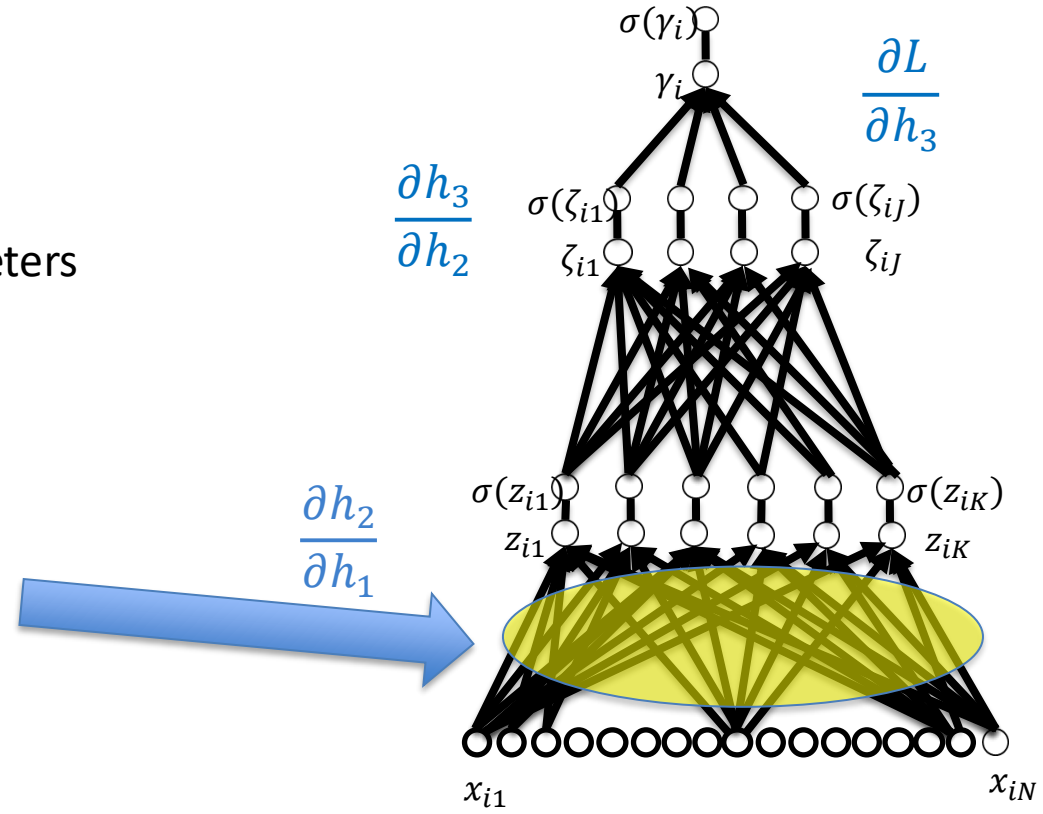
Backward Pass of Gradients, Using h_1, h_2 & h_3

Move backward to parameters at the prior layer

$$(Z_1, b_1)$$

$$h_1 = h[Z_1, b_1, x_i]$$

$$\frac{\partial \ell}{\partial Z_{1,jk}} = \frac{\partial \ell}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial Z_{1,jk}}$$



Backward Pass of Gradients, Using h_1, h_2 & h_3

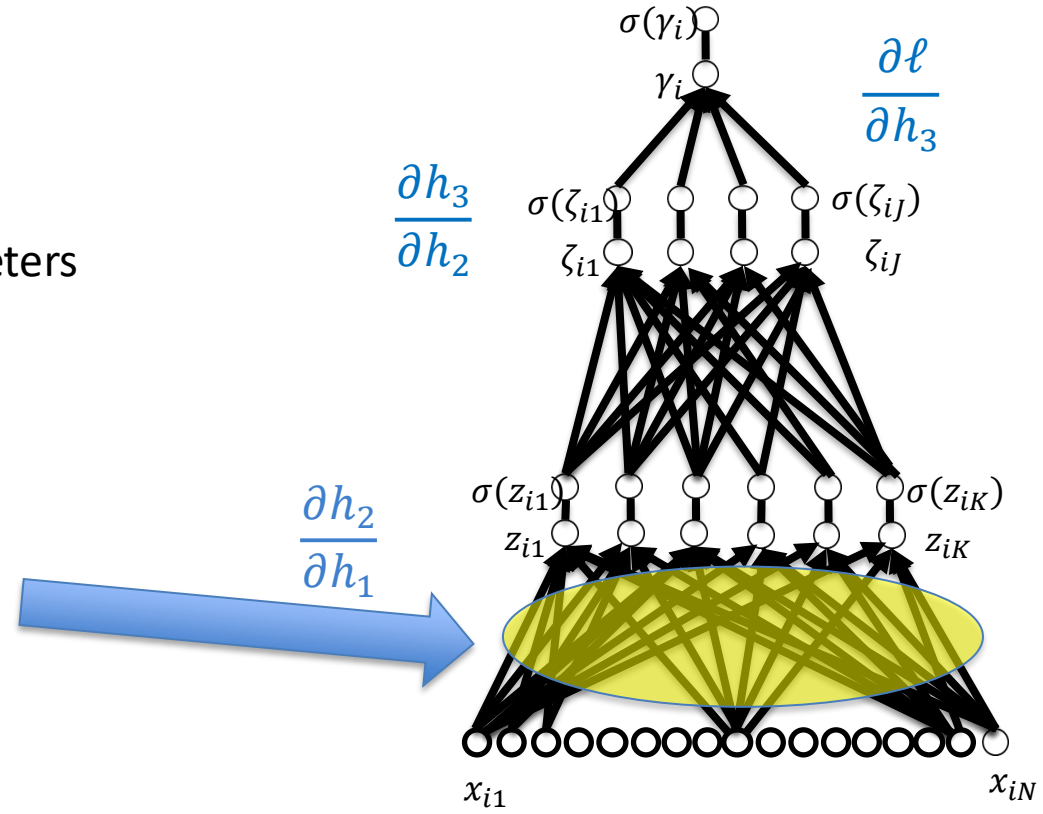
Move backward to parameters at the prior layer

$$(Z_1, b_1)$$

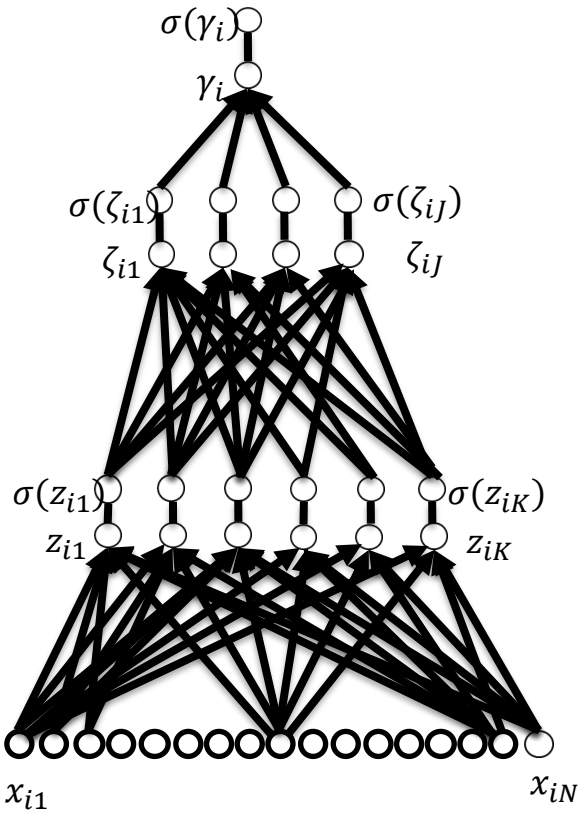
$$h_1 = h[Z_1, b_1, x_i]$$

$$\frac{\partial \ell}{\partial Z_{1,jk}} = \frac{\partial \ell}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial Z_{1,jk}}$$

$$= \frac{\partial \ell}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h[Z_1, b_1, x_i]}{\partial Z_{1,jk}}$$



Backward Pass to Efficiently Perform Gradients



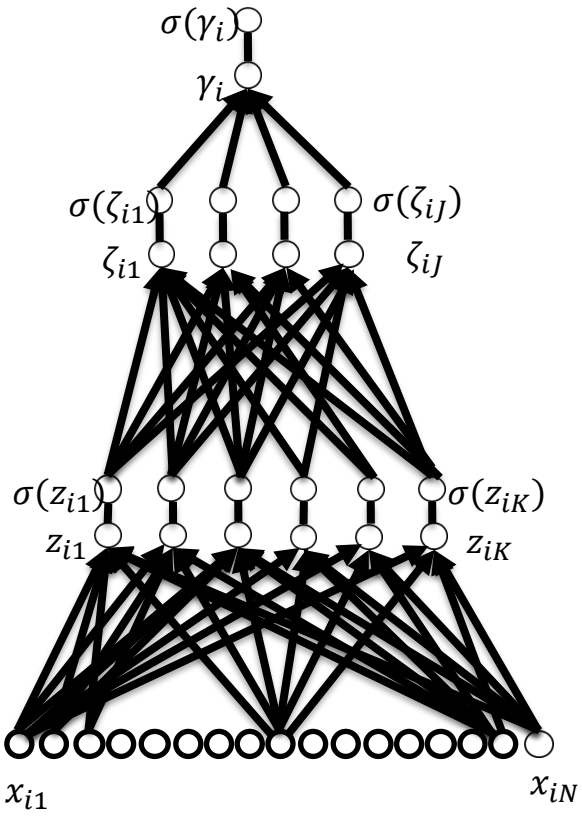
$h_3 \in R$

$h_2 \in R^J$

$h_1 \in R^K$

Backward pass

Back-Propagation Algorithm

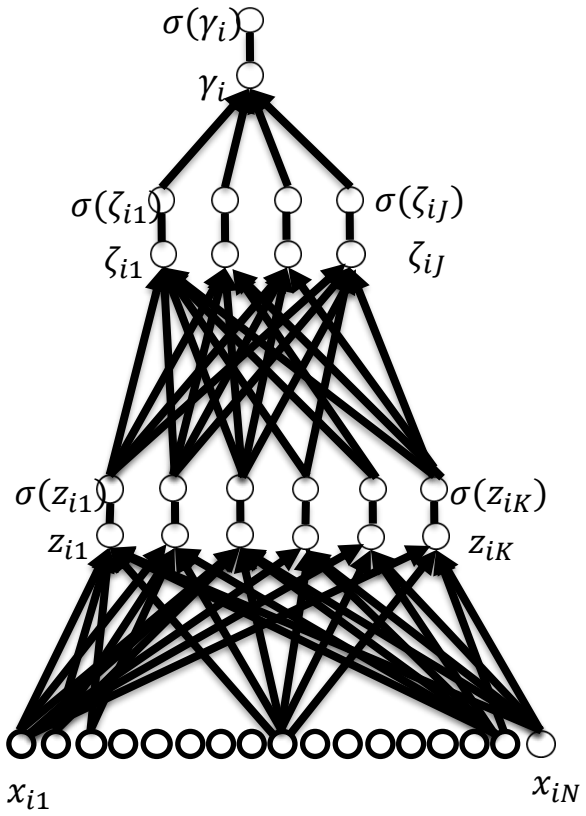


Forward pass

Backward pass

Compute all hidden vectors with θ_k

Back-Propagation Algorithm



Forward pass

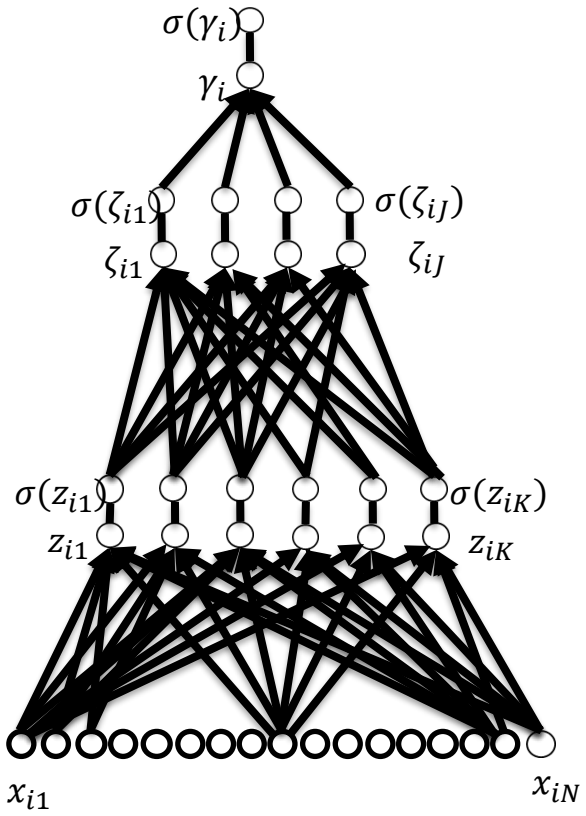
Compute all hidden vectors with θ_k

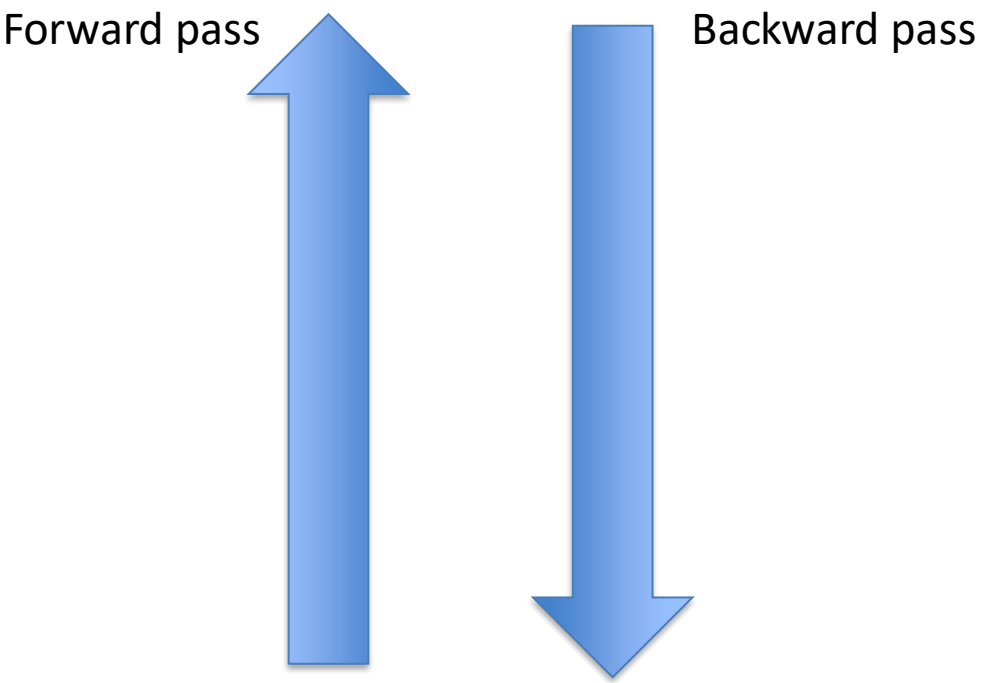
Backward pass

Move backwards performing gradients sequentially, $\theta_k \rightarrow \theta_{k+1}$

Use Chain Rule

Back-Propagation Algorithm



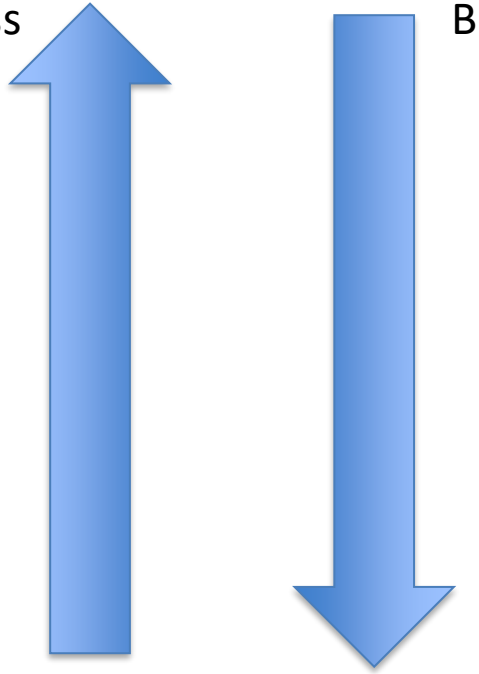


Modern software like PyTorch & TansorFlow performs this
Very efficiently, and automatically

Back-Propagation With Preconditioning

Forward pass

Backward pass



- Backprop efficiently computes gradient g_k
- Result of backprop used in Adam update

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k \odot g_k$$

Additional Technical Matters With NNs

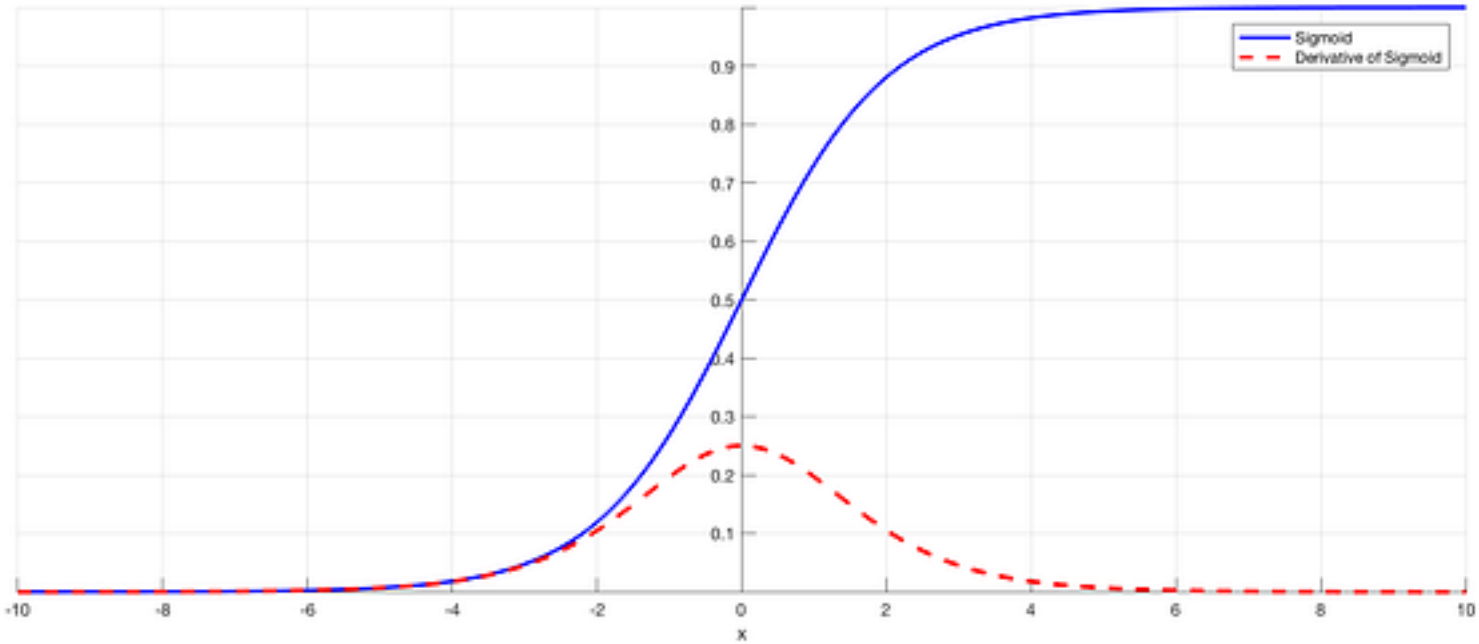
- We see that performing gradients implies a chain of derivatives, e.g.,

$$\frac{\partial \ell}{\partial Z_{1,jk}} = \frac{\partial \ell}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h[Z_1, b_1, x_i]}{\partial Z_{1,jk}}$$

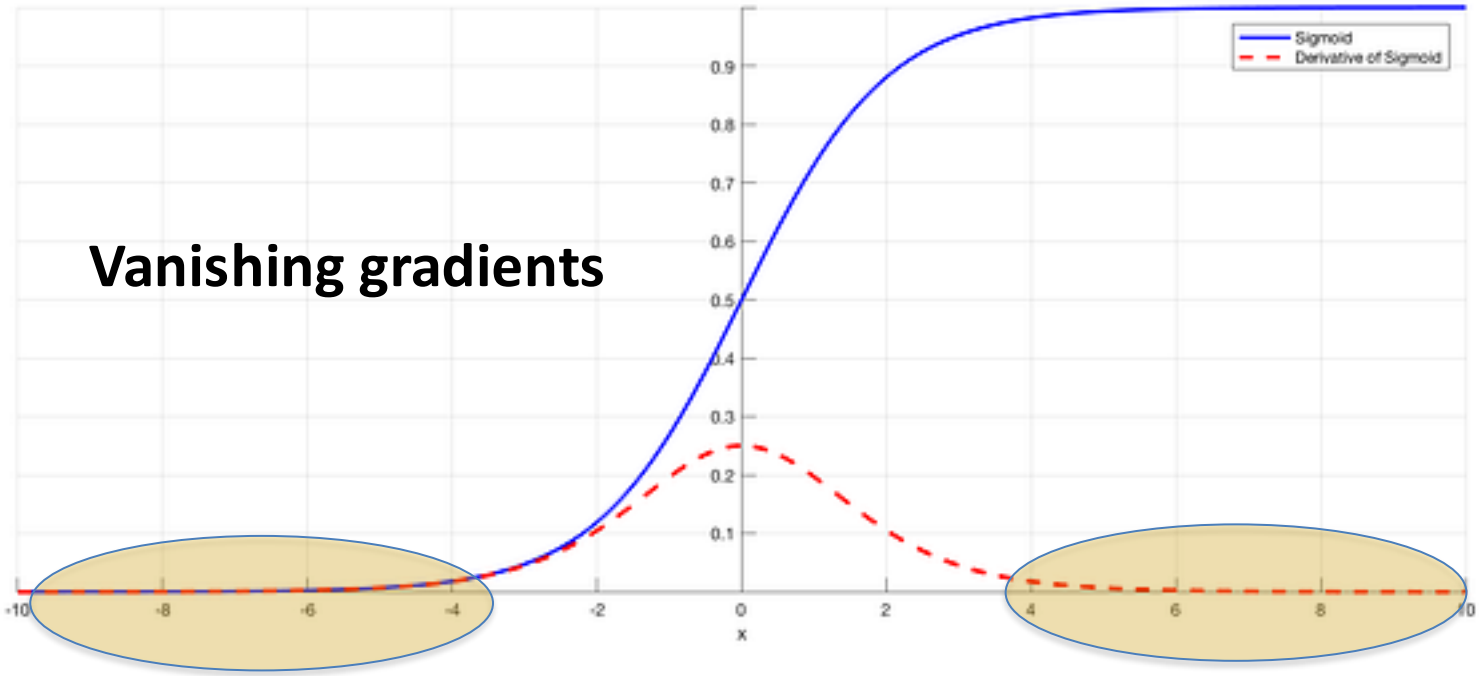
- Recall the sigmoid link function $\sigma(z)$, and consider its derivative $\sigma'(z)$

$$\sigma(z) = 1/[1 + \exp(-z)]$$

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$


Sigmoid
Derivative of Sigmoid
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
-10
-8
-6
-4
-2
0
2
4
6
8
10
x

Vanishing gradients



Limitations of Sigmoid Nonlinearities for Deep NNs

$$\frac{\partial \ell}{\partial Z_{1,jk}} = \frac{\partial \ell}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h[Z_1, b_1, x_i]}{\partial Z_{1,jk}}$$

- Likely that after several layers, sigmoid-based gradients will vanish
- We then lose sensitivity of loss function to parameters deeper in network
- Limits depth of the network

Rectified Linear Unit Nonlinearity

- Recall that the nonlinearity at each layer yields a model that breaks assumption of a linear model

- This has many benefits from the standpoint of general nonlinear function for modeling

- The sigmoidal nonlinearity has connections to logistic regression, and provides intuition, but other nonlinear functions may be considered

- The ReLu unit has gained recent attention: $f(z) = z$ for $z \geq 0$, $f(z) = 0$ for $z < 0$